

LOYALIST COLLEGE

2024S-T4 AIP (Applied Industry Project) Group A

**TBC Student Support Chatbot
Team Chatflix**

Jai Viral Doshi - jaiviraldoshi@loyalistcollege.com

Mayur Rajendra Donwalkar - mayurrajendradonw@loyalistcollege.com

Girish Sai Thiruvadhula - girishsaithiruvadhula@loyalistcollege.com

Keyur Denish Thakkar - keyurdenishthakka@loyalistcollege.com

Deep Amrishkumar Lad - deepamrishkumarla@loyalistcollege.com

Project Advisor: Bhavik Gandhi

Project Co-Ordinator: Stanley Chor

August 01, 2024

TABLE OF CONTENTS

Abstract	7
Team Roles	7
Team Chatfix:	7
Introduction	7
Project Overview:	7
Objectives:	7
Target Audience:	7
Project Scope	7
In-Scope:	7
Out-of-Scope / Future Scope:	8
Project constraints	8
Data Limitations.....	8
Financial Constraints	8
Infrastructure Challenges.....	8
Requirement Clarification	8
Research on approaches	8
Bot Press:	9
Knowledge Base.....	9
Personality Agent.....	9
Summary Agent	10
Intents & Entities	11
Pros & Cons.....	12
Pricing	12
Microsoft Bot Framework	13
Core components	13
High-level architectural diagram	14
How it works?	14
Pricing	14
RASA	15
Features and Functionalities.....	15
How RASA works?.....	16
Advantages	17
Disadvantages.....	17

Pricing	17
Restrictions for Commercial Usage.....	17
RAG (Retrieval augmented generation)	18
What it does?.....	18
RAG Features and Functionalities.....	19
Information Retrieval.....	19
Improved LLM Performance:	19
Additional Functionalities:	19
Pricing	20
Open Source LLM's:	22
Pros and Cons:	23
Reasons for choosing the Llama3 and PgVector	24
LLM Evaluation: Ollama Models	24
Evaluated models	24
Evaluation Results	24
Finalizing llama3	24
Vector Store	24
How it works:.....	25
Key Features:	25
Use Cases:	25
Research on vector store.....	25
FAISS: A LIBRARY FOR EFFICIENT SIMILARITY SEARCH	25
QDRANT	26
PGVECTOR VS. CHROMADB VS. FAISS VS. QDRANT	27
Strengths of PgVector	27
Strengths of ChromaDB	27
Strengths of FAISS.....	27
Strengths of QDRANT	27
PgVector vs. Competitors	28
DSPy: A Framework for RAG Evaluation	28
Key Features of DSPy:.....	28
How DSPy Works:	29
Architecture and Design	29
System Architecture	29

Here are some of the key benefits of using a RAG architecture:.....	30
Design Diagrams	31
ERD: Entity-Relationship Diagram	31
Chatflix Data: Entity-Relationship Diagram	31
Table details	32
ADMIN_TBL:.....	32
Knowledge base.....	33
USERS_TBL	33
Langchain_Pg_Collection	34
Langchain_Pg_Embedding	34
Technology Stack	35
Features and Functionalities	36
Core Features.....	36
Integration and APIs	37
What is an API?.....	37
How does an API work?	37
Why are APIs important?	37
API operations: The building blocks of interactions	38
Common API operations.....	38
Additional operations (less common)	38
Best practices for API operations	38
API development in python.....	39
FASTAPI	39
DJANGO	39
FLASK	39
FLASK API VS FASTAPI VS DJANGO	39
FLASK: Ideal choice for our chatbot prototype.....	40
Integration points.....	40
API testing tools: POST MAN	41
What is postman?	41
Why use POSTMAN?.....	41
APIs developed in project.....	41
Data Management	43
PostgreSQL Data Storage and Security on Local Machine.....	43

Data Storage.....	43
Data Privacy.....	43
Data Backup and Recovery.....	44
User Interface (UI)	44
Chat Interface	44
Admin Panel	45
Security	47
Authentication and Authorization.....	47
How Users Are Authenticated	48
Testing and Quality Assurance	48
Testing Strategies:	48
API:.....	48
Steps to Test the API:	48
Testing Endpoints:	48
Metrics in performance evaluations	49
Evaluating RAG Model Performance	49
Retrieval Component Evaluation	49
Generation Component Evaluation	50
End-to-End Evaluation	50
Opted Evaluation Strategy.....	50
ROUGE score.....	50
How it works:	50
Interpretation:.....	50
Limitations:	51
Response reranking	51
Human Evaluation for RAG Model Assessment.....	51
Developer Guide	51
Data Loading and Modelling.....	51
What is Ollama?	51
Benefits of Using Ollama:	52
Things to Consider:.....	52
Ollama Installation	52
Pulling the Models from Ollama.....	53
Pulling and Running Llama3 with Ollama on Different Operating Systems	53

Additional Notes:	54
Troubleshooting:	55
PostgreSQL and PGVector installation	55
Installing PostgreSQL and pgvector on Different Operating Systems	55
Windows:	56
Enabling pgvector Extension:	56
Verifying Installation:	56
Python & Package installation	57
Python.....	57
Linux:	57
Python Package Installation	58
Backend Development	58
Development Environment	58
Frontend Development.....	59
Development Environment	59
Installing HTML, CSS, and JavaScript: A Misconception	59
What you need:	59
How to start:	59
Installing XAMPP on Different Operating Systems	60
What is XAMPP?.....	60
Installation Steps.....	60
Additional Considerations	61
Project Management	61
Project Timeline - The Original	62
Project Timeline - Altered Timeline	63
Key Changes:	63
Team Structure for Chatbot Development	64
Role Breakdown	64
Risk Management	65
Timeline Adherence:	65
Use Case Changes:	65
Portfolio Website	65
References and Resources	65

ABSTRACT

The TBC College Student Support Chatbot is designed to enhance the student experience by providing real-time assistance and information. Leveraging natural language processing and machine learning, the chatbot offers personalized support, answers queries, and assists students through various college processes, thereby improving accessibility and efficiency.

TEAM ROLES

TEAM CHATFLIX:

- *Girish Sai Thiruvithula*: Project Manager and AI Modeling Engineer
- *Mayur Donwalkar*: Frontend Developer
- *Keyur Thakkar*: Data Scientist
- *Deep Lad*: AI Engineer
- *Jai Doshi*: AI Engineer

INTRODUCTION

PROJECT OVERVIEW:

The TBC College Student Support Chatbot is an AI-powered virtual assistant designed to provide students with immediate access to information and support. Its primary functions include answering inquiries related to a comprehensive resource guide for international students, covering safety, support services, campus life, and policies, including essential information on scam prevention, human rights, sexual assault, and accessibility.

OBJECTIVES:

- Enhance student support services by providing 24/7 assistance.
- Reduce the workload on administrative staff by automating routine queries.
- Improve student satisfaction and engagement through personalized interactions.
- Ensure seamless integration with existing college systems.

TARGET AUDIENCE:

The primary users of the chatbot are TBC College students. Secondary users include prospective students, faculty, and administrative staff.

PROJECT SCOPE

IN-SCOPE:

The chatbot will be used to assist in logging complaints related to various aspects of college life, including academic issues, housing, college services, and facilities [refer the project overview]. This project encompasses the analysis, design, development, and testing of a chatbot prototype,

along with comprehensive documentation serving as a proof-of-concept. The chatbot's functionalities will be supported to the extent possible by data provided by TBC stakeholders.

OUT-OF-SCOPE / FUTURE SCOPE:

- Directly handling complaints.
- Providing information related to events.
- Utilizing URL, Word, and Text documents as data sources.
- Directly updating existing documents.
- Saving user chat history.

PROJECT CONSTRAINTS

DATA LIMITATIONS

Access to certain student information is restricted due to privacy regulations and the need to comply with GDPR and other data protection laws.

FINANCIAL CONSTRAINTS

Training in the necessary Large Language Models (LLMs) incurs significant costs. To achieve optimal performance and speed, substantial infrastructure and premium models are required.

INFRASTRUCTURE CHALLENGES

The implementation of a scalable and reliable chatbot platform necessitates the utilization of cloud services.

REQUIREMENT CLARIFICATION

Obtaining precise and concrete project specifications from the college has been challenging. There has been considerable back-and-forth regarding the specific expectations of TBC.

RESEARCH ON APPROACHES

Implementing a question-answering chatbot can be achieved through various methods. The following approaches were considered:

- Bot Press
- Microsoft Bot Framework
- RASA
- RAG - DSPy
- RAG - Langchain

Let us discuss the above approaches, along with pros and cons in detail:

BOT PRESS:

Botpress is a chatbot platform to create, manage and publish chatbots. With Botpress you can create chatbots and launch these on multiple channels, such as your website, Facebook, and WhatsApp. Key features of Botpress include:

- Flow builder to create chat automation.
- Built-in Artificial Intelligence, powered by ChatGPT.
- Publish your chatbot on multiple channels, including your website, Facebook, Instagram, WhatsApp, Telegram, Slack and more.

This is where Botpress sets itself apart from the competition. Botpress has extremely advanced AI capabilities.

In Botpress, you have multiple AI agents that you can enable for each bot:

- Knowledge Agent
- Personality Agent
- Summary Agent
- Translator Agent (Automatically translate each message sent to a user)

KNOWLEDGE BASE

Upload knowledge base sources like Websites, Tables, and Documents (PDF, HTML, doc and txt), search the web, and rich text (Type knowledge directly in a simple text editor).

PERSONALITY AGENT

Using the Personality Agent, you can give Botpress instructions on the bot's personality. Then, Botpress will automatically rewrite every message that is sent to a user so that it matches the described personality:

Personality Agent

v0.1

Configuration

About

Enable Agent

Enabling the agent allows it to actively participate in conversations. When enabled, the agent will process user input, generate responses, and take actions as configured.

☒

Configuration

Enable Personality Rewriting

☒

Personality Description

Describe what your chatbot is meant to do and how it should behave. You can include some personality traits here as well to influence how the chatbot will respond. Expressions are supported.

You are Ali Abdaal. A former doctor turned YouTuber, entrepreneur, podcaster and soon-to-be author. I've been creating YouTube videos for over 7 years and have amassed a following of over 4 million on my main channel.

Description of Ali Abdaal:

My YouTube journey started back in 2017 when I was still a student at Cambridge University. I was studying to become a doctor (the dream was to specialise in plastic surgery). Back then, I started creating videos to help other people with studying and passing exams. This is when I started my first business with my brother, 6Med. We created a platform to help people pass the entrance exams and get into medicine.

Fast forward a few years and the videos I was creating started to be geared more towards productivity and passing on the knowledge that got me through medical school. I'd spend every spare minute I had planning the next video, writing scripts or researching ideas.

Exposed Variables

No variables exposed for this agent

Reset to default values

SUMMARY AGENT

You can automatically summarize a complete conversation with the agent. This is great when you are handing over the conversation to an agent, so that person knows immediately what the conversation is about:

Summary Agent

v0.1

Configuration

About

Enable Agent

Enabling the agent allows it to actively participate in conversations. When enabled, the agent will process user input, generate responses, and take actions as configured.

☒

Configuration

Summary Max Tokens

The maximum length of the generated summary (in tokens)

100

Transcript Max Lines

The number of lines to include in the transcript. (0 to disable)

10

Exposed Variables

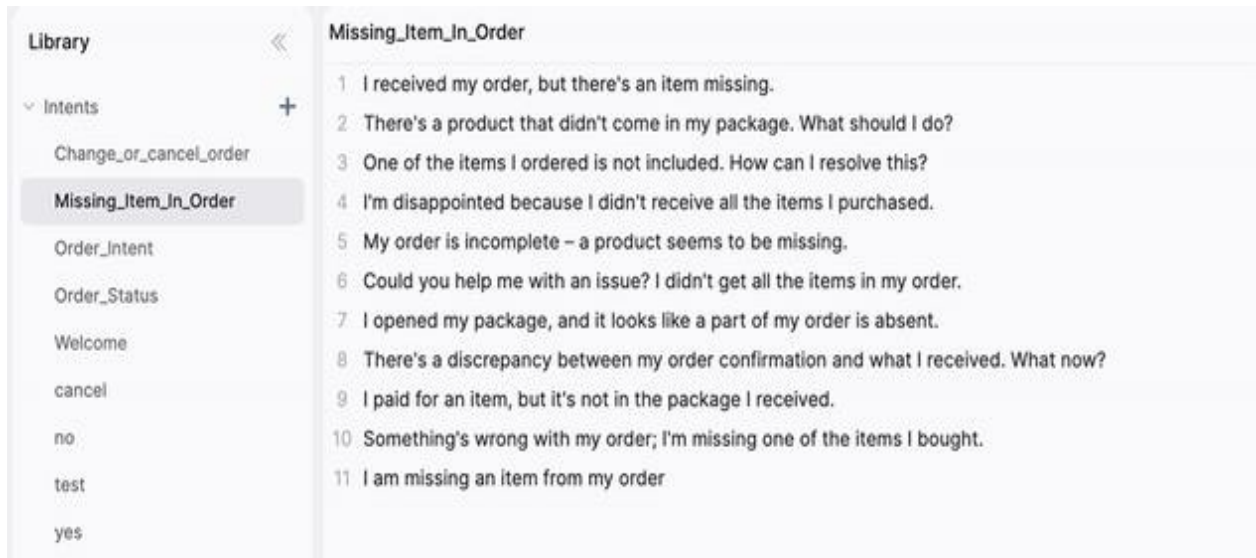
Tt {{conversation.SummaryAgent.summary}}

Tt {{conversation.SummaryAgent.transcript}}

Reset to default values

INTENTS & ENTITIES

Aside from the agents, you can also create Intents in Botpress to understand the intent of the user based on training phrases you provide:



The screenshot shows the Botpress interface for creating an intent. On the left, a 'Library' sidebar lists various intents, with 'Missing_Item_In_Order' selected. The main area displays the training phrases for this intent, numbered 1 through 11.

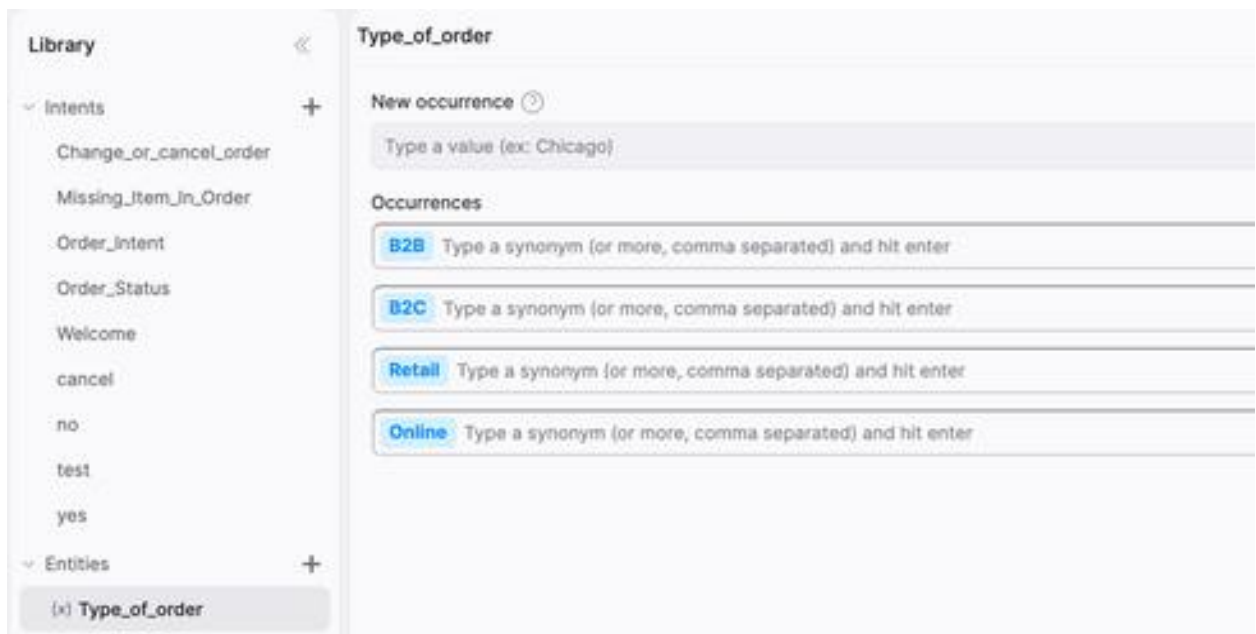
Library

- Intents
 - Change_or_cancel_order
 - Missing_Item_In_Order**
 - Order_intent
 - Order_Status
 - Welcome
 - cancel
 - no
 - test
 - yes

Missing_Item_In_Order

- 1 I received my order, but there's an item missing.
- 2 There's a product that didn't come in my package. What should I do?
- 3 One of the items I ordered is not included. How can I resolve this?
- 4 I'm disappointed because I didn't receive all the items I purchased.
- 5 My order is incomplete – a product seems to be missing.
- 6 Could you help me with an issue? I didn't get all the items in my order.
- 7 I opened my package, and it looks like a part of my order is absent.
- 8 There's a discrepancy between my order confirmation and what I received. What now?
- 9 I paid for an item, but it's not in the package I received.
- 10 Something's wrong with my order; I'm missing one of the items I bought.
- 11 I am missing an item from my order

Additionally, Botpress has lots of built-in entities and you can create our entities to capture specific information from a sentence:



The screenshot shows the Botpress interface for creating an entity. On the left, a 'Library' sidebar lists various entities, with 'Type_of_order' selected. The main area displays the configuration for this entity, including a 'New occurrence' field and a list of 'Occurrences'.

Library

- Intents
 - Change_or_cancel_order
 - Missing_Item_In_Order
 - Order_intent
 - Order_Status
 - Welcome
 - cancel
 - no
 - test
 - yes
- Entities
 - (x) Type_of_order**

Type_of_order

New occurrence ⓘ

Type a value (ex: Chicago)

Occurrences

- B2B** Type a synonym (or more, comma separated) and hit enter
- B2C** Type a synonym (or more, comma separated) and hit enter
- Retail** Type a synonym (or more, comma separated) and hit enter
- Online** Type a synonym (or more, comma separated) and hit enter

PROS & CONS

Advantages	Disadvantages
Open-Source Nature	Learning Curve (Requires technical expertise to implement and maintain)
Flexibility and Customization (Open-source platforms allow for tailoring the chatbot to specific needs and preferences.)	Documentation and Support
Scalability	Maintenance and Updates (Ongoing efforts are needed to keep the chatbot up-to-date and secure.)
Multi-Channel Support	Resource Intensive
Rich Features	Dependency on Community
Strong Community Support	Data should be given to a third-party vendor

PRICING

FREE PLAN	ENTERPRISE - USD 495/Month	CUSTOM
5 bots	20 bots (with Always Alive functionality)	Bots
2,000 incoming messages & events/month	250,000 incoming messages & events/month	Always Alive
3 collaborators	5 collaborators	Incoming messages & events
5,000 Table rows	1,000,000 Table rows	Collaborators
100MB vector database storage	2GB vector database storage	Table Rows
File Storage – 500 MB	File Storage – 10 GB	Custom
Data Retention – 30 Days	Data Retention – 90 Days	Custom

API Calls – 100 API Calls/s	API Calls – 1000 API Calls/s	Custom
\$5 AI Credit	N/A	Vector DB storage (GB)
Community support	Live chat support	Custom

NOTE: As it is a 3rd party tool, the prices may vary depending on their business plan.

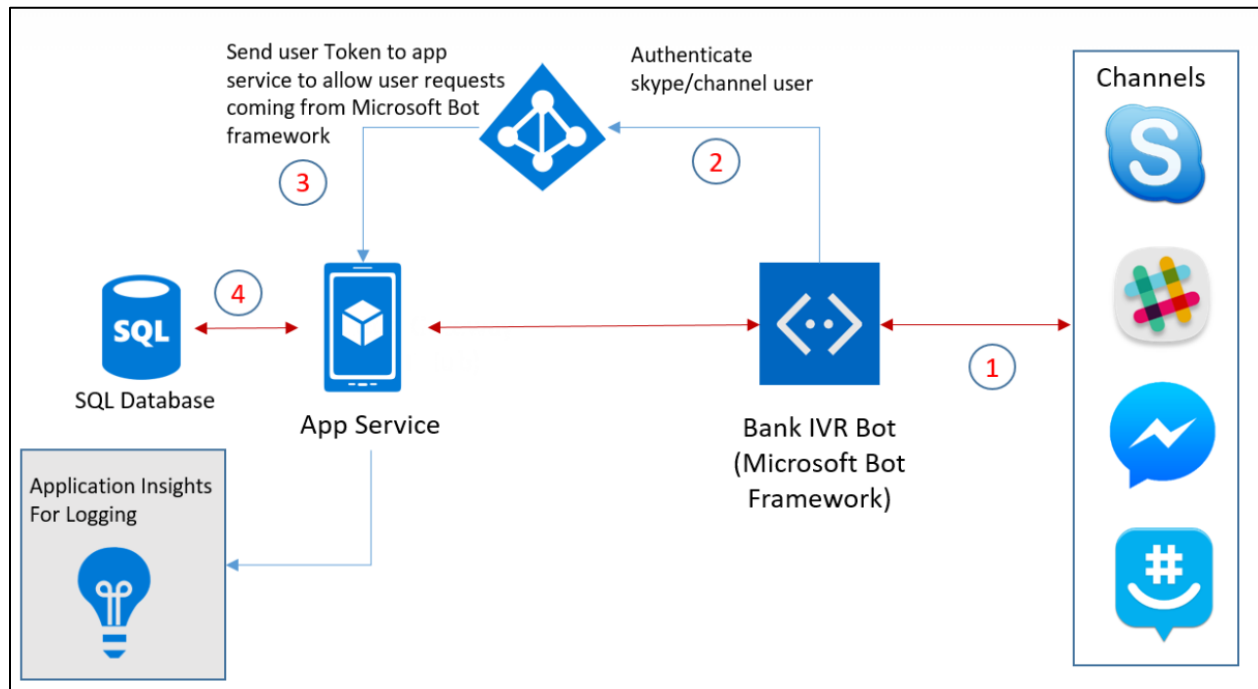
MICROSOFT BOT FRAMEWORK

Microsoft Bot Framework and Azure AI Bot Service are a collection of libraries, tools, and services that let you build, test, deploy, and manage intelligent bots. The Bot Framework includes a modular and extensible SDK for building bots and connecting to AI services. With this framework, developers can create bots that use speech, understand natural language, answer questions, and more.

CORE COMPONENTS

- **Bot Builder SDKs:** The framework offers SDKs for different programming languages (primarily C# and JavaScript/Node.js) to build bots. These SDKs help developers handle dialogs, manage state, and integrate with external APIs.
- **Bot Service:** This is the primary azure service for deploying and managing bots. It provides a scalable environment and integrates seamlessly with Azure Functions, Cognitive Services, and other Azure resources.
- **Channels:** The Bot Framework supports multiple channels, allowing bots to interact with users on various platforms such as Microsoft Teams, Facebook Messenger, Slack, Skype, and more.
- **Azure Bot Service:** This service simplifies bot creation with a serverless, scalable architecture. It provides tools for bot development, testing, and deployment.
- **QnA Maker and LUIS:** These are AI services that enhance bots' capabilities. QnA Maker helps build bots that can answer questions based on FAQs, while LUIS (Language Understanding) enables natural language understanding for better user interactions.

HIGH-LEVEL ARCHITECTURAL DIAGRAM



HOW IT WORKS?

- 1. User Interaction:** A customer (the user) places an order (sends a message).
- 2. Order Received:** The shipping department (Bot Connector Service) receives the order and passes it to the assembly line (Bot Builder SDK).
- 3. Order Processing:** The assembly line (Bot Builder SDK) examines the order, consults the language expert (LUIS) to understand the request, and if needed, checks the knowledge base (QnA Maker) for an answer.
- 4. Order Fulfillment:** The assembly line creates a response and sends it back to the shipping department.
- 5. Delivery:** The shipping department (Bot Connector Service) delivers the response to the customer.

PRICING

AZURE BOT SERVICE:

	Free	\$1
Standard channels	Unlimited messages	Unlimited messages
Premium channels	10,000 messages/month	\$0.50 per 1,000 messages

COGNITIVE SERVICES:

LUIS: Free tier available with up to 5,000 transactions per month. Standard pricing is \$1.50 per 1,000 transactions.

Language service can be purchased in tiers.

Text Records correspond to the number of 1,000-character units.

1. Summarization features count the combined length of input document and corresponding output summary as text records. Other Language features count the length of input document.
2. Summarization, sentiment analysis, key phrase extraction, language detection, question answering, named entity recognition, and conversational language understanding all share 5,000 free text records per month.
3. Please contact sales for pricing of over 10 million text records.
4. Some features such as question answering require dependent resources. When you choose question answering, an Azure AI Search resources will also be deployed in your subscription.
5. Custom Text Analytics for health (preview) is limited to 5,000 free text records per language resource, see region support to apply for a higher quota, please submit an Azure support ticket.

QnA Maker: Free tier available. Standard pricing depends on the usage and features.

Instance	Transactions Per Second (TPS)	Features	Price
Free	3 TPS	Up to 1MB each document Up to 100 transactions per minute Up to 50,000 transactions per month	3 managed documents free per month
Standard	3 TPS	Up to 100 transactions per minute	\$10 for unlimited managed documents

RASA

RASA is an essential open-source machine learning framework used to build customized AI chatbots. It can be integrated with websites, Facebook, Telegram, WhatsApp, Slack, Discord, etc.

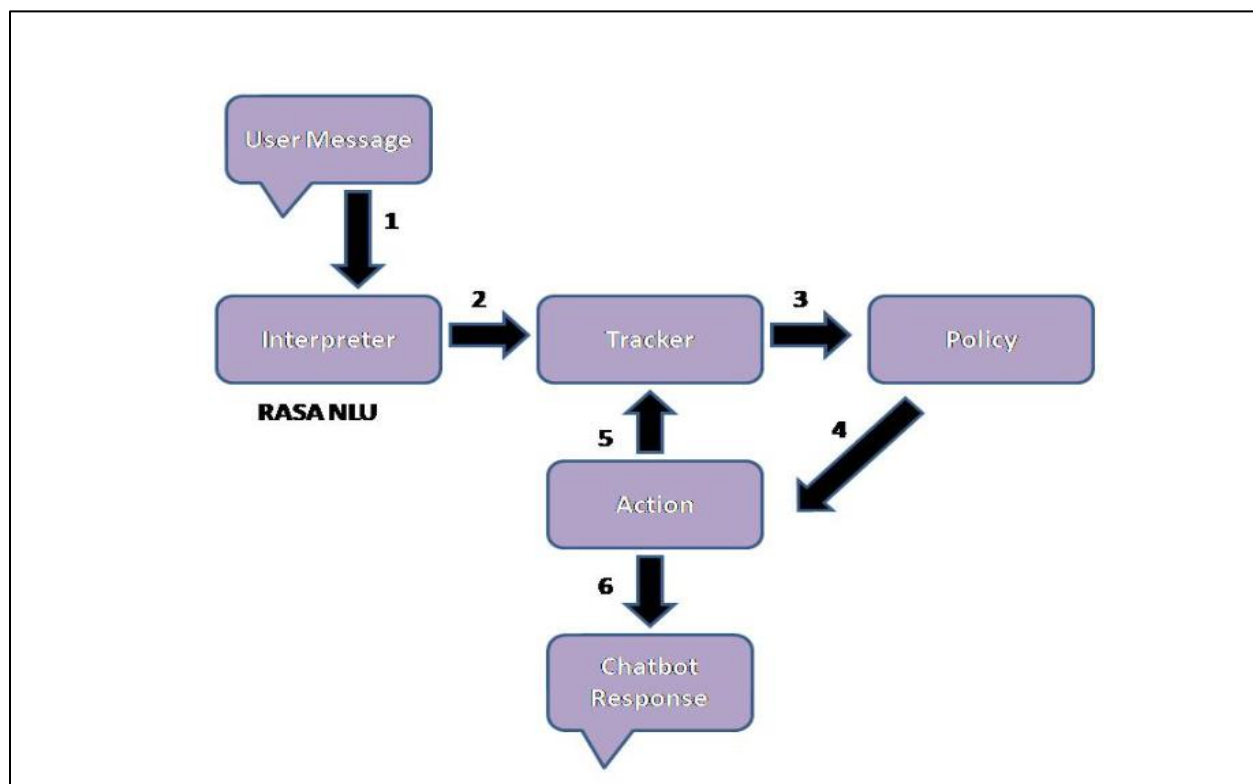
RASA has two main components: RASA NLU and RASA CORE. RASA NLU interprets the user input and extracts entities and intent with the help of various pipelines. It then converts the input into a dictionary that includes the original text, intent, and entities identified, which is then sent to RASA CORE. RASA CORE is responsible for the chatbot's response. It selects the appropriate response as per the user input and then sends it back as a chatbot response. RASA also offers RASA X functionality which provides a Web UI and supports interactive learning.

FEATURES AND FUNCTIONALITIES

- **Natural Language Understanding (NLU):** RASA can interpret user messages, identify intents, and extract relevant entities.
- **Dialogue Management:** It manages conversation flow, tracks dialogue state, and determines appropriate actions based on user inputs.

- **RASA Core:** This component handles the dialogue management. It decides what action to take based on the user's message and the current conversation state.
- **RASA X:** This is the graphical user interface (GUI) for building, testing, and deploying conversational AI assistants.
- **Machine Learning:** RASA incorporates machine learning models for improving NLU and dialogue management over time.
- **Rasa SDK:** Allows customization of actions performed by the chatbot. Actions can include querying databases, calling APIs, or performing any other task required to fulfil user requests.
- **Customizable:** Allows developers to create tailored chatbots with specific functionalities and personalities.
- **Integration:** Can be integrated with various messaging platforms and backend systems.
- **Open Source:** Offers flexibility and control over the chatbot development process.

HOW RASA WORKS?



- User message is sent to the interpreter, which converts the message into a dictionary having original text, the intent, and entities. This is handled by RASA NLU.
- The interpreter sends the message to Tracker which keeps a record of the conversation.
- The current state of the Tracker is sent to each policy.
- The policy then chooses which action must be taken next.
- Action chosen is recorded in the Tracker.
- Chatbots Response sent as per the chosen action.

ADVANTAGES

- **Flexibility:** RASA's open-source nature allows for customization to fit specific use cases.
- **Cost-Effective:** Being open source, there are no licensing fees.
- **Control:** Developers have full control over the chatbot's logic and data.
- **Scalability:** Can handle increasing conversation volumes and complexity.
- **Community Support:** A strong community provides resources and assistance.

DISADVANTAGES

- **Development Effort:** Building complex chatbots requires technical expertise.
- **Initial Setup:** Setting up RASA can be time-consuming for beginners.
- **Maintenance:** Requires ongoing maintenance and updates.
- **Lack of Pre-built Components:** Unlike some proprietary platforms, RASA might require more development for common chatbot features.

PRICING

RASA is open source, meaning there is no direct pricing. However, there might be costs associated with:

- **Infrastructure:** Hosting the chatbot application, database, and machine learning models.
- **Development Resources:** Time spent on development, testing, and maintenance.
- **Third-Party Services:** Integration with external services like payment gateways or messaging platforms might incur costs.
- **Enterprise:** For larger organizations with specific needs, RASA offers an enterprise subscription. This typically includes:
 - ❖ Company license
 - ❖ Analytics
 - ❖ Role-based access control
 - ❖ Multiple deployment environments
 - ❖ Single Sign-On
 - ❖ Service Level Agreements
 - ❖ Deployment and installation support
 - ❖ Expert support

NOTE: The exact pricing for the enterprise subscription would depend on the specific needs of your organization and would require direct contact with RASA for a quote.

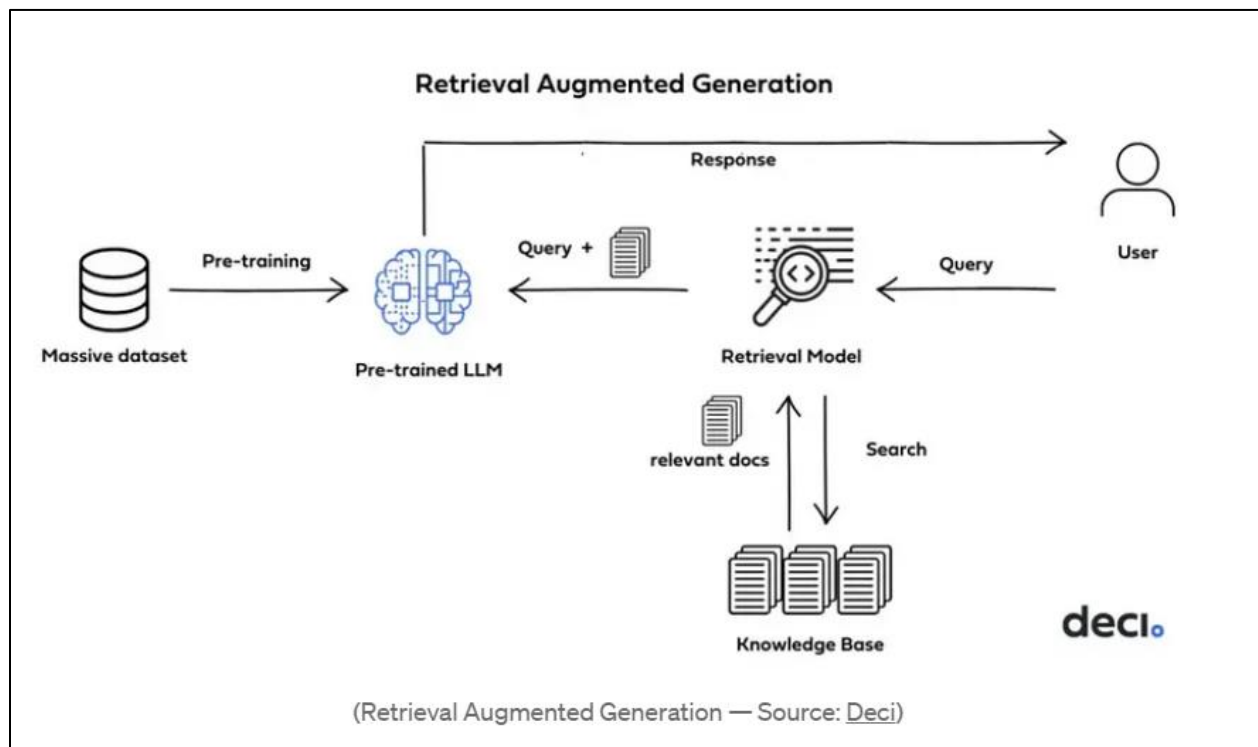
RESTRICTIONS FOR COMMERCIAL USAGE

There are no specific restrictions on using RASA for commercial purposes. As an open-source project, you are free to use it for any application. However, it's essential to comply with any third-party libraries or services used within our chatbot.

RAG (RETRIVEAL AUGMENTED GENERATION)

RAG stands for Retrieval-Augmented Generation. It's a technique that enhances the capabilities of large language models (LLMs) by combining them with a retrieval system. This allows the model to access and process information from external knowledge sources in real-time, making its responses more accurate, relevant, and up to date. Here's a breakdown:

- **Query Processing:** The user submits a query or prompt.
- **Retrieval:** The system retrieves relevant information from an external knowledge base based on the query.
- **Augmentation:** The retrieved information is combined with the original query and fed into the LLM.
- **Generation:** The LLM generates a response based on the combined information.



WHAT IT DOES?

- **Fact-checks:** RAG consults external knowledge bases before generating a response. This ensures the information you receive is grounded in real-world data.
- **Ups the Accuracy Game:** By incorporating external information, RAG helps LLMs avoid making claims based solely on their training data, which can sometimes be incomplete or outdated.
- **Context is King:** RAG considers the specific context of our query when searching for relevant information, leading to more focused and relevant responses.

- **Think of it this way:** Imagine an LLM as a student who has been studying a massive textbook. RAG acts like a library visit, allowing the student to consult additional resources for a more well-rounded understanding before answering our question.

RAG FEATURES AND FUNCTIONALITIES

RAG (Retrieval-Augmented Generation) focuses on enhancing the capabilities of Large Language Models (LLMs) by incorporating external information retrieval. Here is a breakdown of its key features and functionalities:

INFORMATION RETRIEVAL

- **External Knowledge Sources:** RAG can connect to various external databases, knowledge graphs, or even live feeds to access up-to-date information. This allows LLMs to go beyond their training data and provide responses based on real-world facts.
- **Contextual Search:** RAG does not simply throw information at the LLM. It retrieves information relevant to the specific context of the user's query. This ensures the generated response is focused and addresses the user's specific needs.
- **Scalability:** RAG can be easily adapted to integrate with added information sources as needed. This makes it ideal for situations where access to fresh data is crucial.

IMPROVED LLM PERFORMANCE:

- **Enhanced Accuracy:** By leveraging external data, RAG helps LLMs avoid generating responses based on potentially inaccurate or outdated information in their training data. This leads to more reliable and trustworthy responses.
- **Reduced Hallucinations:** LLMs can sometimes fabricate information, also known as hallucinations. RAG mitigates this risk by grounding the LLM's output in real-world data.
- **Context-Aware Generation:** RAG considers the specific context of the query when generating text. This allows for more nuanced and relevant responses compared to LLMs that solely rely on their internal knowledge base.

ADDITIONAL FUNCTIONALITIES:

- **Integration with LLMs:** RAG can work with various LLM architectures, making it a versatile tool for developers.
- **Fine-tuning:** RAG models can be fine-tuned to improve performance for specific domains or tasks. This allows for customization based on our specific needs.
- **Source Attribution (Optional):** Some RAG implementations allow the generated response to include citations or references to the retrieved information sources. This can enhance user trust and transparency.

PRICING

Unfortunately, RAG itself does not have a fixed pricing structure like traditional software. Here's why:

- **Not a Standalone Product:** RAG is a technique, not a commercially available product. It's often integrated into existing LLM platforms or custom-built solutions.
- **Implementation Costs:** The cost of using RAG depends on how it's implemented. Factors like infrastructure, chosen LLM platform, and the complexity of information retrieval systems can influence the overall cost.
- **Development vs. Cloud Services:** Building a custom RAG solution in-house requires developer expertise and infrastructure investment. Alternatively, some cloud-based LLM services might offer RAG functionalities as part of their pricing structure.

During our work and research below are the best LLM's where we can invest our money.
OpenAI: Open AI charges the users based on tokens both for LLM usage and embeddings.

LLM COST

GPT-4o

GPT-4o is our most advanced multimodal model that's faster and cheaper than GPT-4 Turbo with stronger vision capabilities. The model has 128K context and an October 2023 knowledge cutoff.

[Learn about GPT-4o ↗](#)

Model	Pricing	Pricing with Batch API*
gpt-4o	\$5.00 / 1M input tokens \$15.00 / 1M output tokens	\$2.50 / 1M input tokens \$7.50 / 1M output tokens
gpt-4o-2024-05-13	\$5.00 / 1M input tokens \$15.00 / 1M output tokens	\$2.50 / 1M input tokens \$7.50 / 1M output tokens

Vision pricing calculator

Set width

150px

by

Set height

150px

= \$0.001275

☐ Low resolution

*Batch API pricing requires requests to be submitted as a batch. Responses will be returned within 24 hours for a 50% discount. [Learn more about Batch API ↗](#)

GPT-3.5 Turbo

The GPT-3.5 Turbo model is our fast, inexpensive model for simple tasks. It has a 16K context window and is optimized for dialog. The Instruct model supports a 4K context window.

[Learn about GPT-3.5 Turbo ↗](#)

Model	Pricing	Pricing with Batch API*
gpt-3.5-turbo-0125	\$0.50 / 1M input tokens	\$0.25 / 1M input tokens
	\$1.50 / 1M output tokens	\$0.75 / 1M output tokens
gpt-3.5-turbo-instruct	\$1.50 / 1M input tokens	\$0.75 / 1M input tokens
	\$2.00 / 1M output tokens	\$1.00 / 1M output tokens

*Batch API pricing requires requests to be submitted as a batch. Responses will be returned within 24 hours for a 50% discount. [Learn more about Batch API ↗](#)

EMBEDDING MODELS COST:

Embeddings are used to convert our textual data into machine understandable format mainly into numerical and these Embedding models are trained on a large chunks of internet data to have context on all the topics of data.

Embedding models

Build advanced search, clustering, topic modeling, and classification functionality with our embeddings offering.

[Learn about embeddings ↗](#)

Model	Pricing	Pricing with Batch API*
text-embedding-3-small	\$0.02 / 1M tokens	\$0.01 / 1M tokens
text-embedding-3-large	\$0.13 / 1M tokens	\$0.07 / 1M tokens
ada v2	\$0.10 / 1M tokens	\$0.05 / 1M tokens

*Batch API pricing requires requests to be submitted as a batch. Responses will be returned within 24 hours for a 50% discount. [Learn more about Batch API ↗](#)

VERTEX AI

Vertex AI Agents Chat*	\$12.00 / 1,000 query**
Vertex AI Agents Voice*	\$0.002 / second†
Dialogflow CX Chat*** (Standard, non-Generative AI capabilities for Chat)	\$7.00 / 1,000 query**
Dialogflow CX Voice*** (Standard, non-Generative AI capabilities for Voice)	\$0.001 / second†

<https://cloud.google.com/generative-ai-app-builder/pricing>

All the above-mentioned LLM's are closed source and premium LLM's. On the other hand, open source LLMs can be used in colleges for research, education, customization, and fostering innovation through transparency and community collaboration.

OPEN SOURCE LLM'S:

On the internet there are a lot of opensource LLMs tuned for the domain specific needs and based on their training time and model architecture LLM's accuracy depends on.

While both Llama2 and Llama3 are not technically open source in the strictest sense, they are freely available for research and commercial use with some limitations, and they are giving accurate results during our research and work. Here is what you need to know:

LLAMA2

- **Availability:** Freely available for download and use with weights and starting code accessible on GitHub.
- **Limitations:** Organizations with over 700 million monthly active users need to request a license from Meta (which may or may not be granted).
- **Access:** Also available through various providers like Hugging Face, Azure, and AWS.

LLAMA3

- **Availability:** Like Llama2, freely available with weights and starting code accessible.
 - **Limitations:** Same limitation for organizations exceeding 700 million monthly active users.
 - **Access:** Accessible through various cloud platforms like Google Cloud, Databricks, Snowflake, and others, along with providers like Hugging Face and Kaggle.
- Here are some resources to get you started with Llama2 and Llama3:
- **Meta Llama Website:** <https://llama.meta.com/>
 - **Hugging Face:** <https://huggingface.co/docs/transformers/en/index> (Look for models containing "Llama")
 - **Cloud Provider Platforms:** Search for "Llama" models on our preferred cloud provider platform (e.g., AWS Marketplace, Azure Machine Learning)

NOTE:

- Using these models might require some technical expertise depending on our chosen access method (downloading code vs. cloud platforms).
- While freely available, you might incur costs depending on our chosen access method (e.g., cloud platform usage fees).

PROS AND CONS:

Below table gives the information regarding the advantages and disadvantages among RAG, RASA, and Bot Press:

Technology	Function	Pros	Cons
RAG	Improves LLMs for contextual responses	Enhanced Accuracy, Scalability, Context-aware Outputs	Complexity, Resource Intensive, Data Dependence
RASA & Botpress	Chatbot development platforms	Simplified Development, Customization, Integration	Limited AI Capabilities, Domain Expertise, Vendor Lock-in

Below table gives the information regarding the data, privacy and cost among RAG, RASA and Bot-press

Features		RAG	RASA & Botpress
Data	Advantages	Leverages external data for improved accuracy and factual grounding.	No inherent advantage. Relies on quality of training data.
	Disadvantages	Requires access to and management of potentially sensitive external data.	Limited ability to handle out-of-domain requests or data not included in training data.
Privacy	Advantages	Can be configured to anonymize or obfuscate retrieved data before feeding it to the LLM.	No inherent privacy features. User privacy depends on implementation.
	Disadvantages	Security breaches in external data sources could expose sensitive information.	Can be vulnerable to data leakage if not implemented securely.
Cost	Advantages	Potentially lower ongoing costs if	Generally lower initial cost due to readily available open-source platforms.

		leveraging free external data sources.	
	Disadvantages	Higher setup costs if building a custom RAG solution or using paid cloud-based LLM services with RAG.	Potential ongoing costs for server maintenance, custom development, or cloud platform usage.

Based on the pros and cons, we elected RAG model architecture to use it for our project purposes.

REASONS FOR CHOOSING THE LLAMA3 AND PGVECTOR

LLM EVALUATION: OLLAMA MODELS

During our project research, various LLMs available on the Ollama platform were evaluated for their suitability. This section outlines the models tested and their comparative performance.

EVALUATED MODELS

- Gemma2
- Gemma
- Qwen2
- Qwen
- Phi3
- Phi
- Mistral
- Dolphin-Mistral
- Llama3

EVALUATION RESULTS

While all models exhibited varying capabilities, **Llama3** consistently demonstrated superior performance across a range of tasks and prompts within our project scope. Its responses were more comprehensive, relevant, and coherent compared to the other models tested.

FINALIZING LLAMA3

Based on the evaluation, Llama3 is recommended as the primary LLM for our project due to its exceptional performance. Further exploration of Llama3's capabilities and potential fine-tuning is warranted to maximize its utility within the project context.

To know me details, refer the below GitHub URL :

https://github.com/deeplcit/tbc_application_1/tree/feature_Girish/research

VECTOR STORE

A vector store is a specialized database designed to efficiently store and retrieve vector representations of data.

HOW IT WORKS:

- **Data Conversion:** Text, images, or other data are converted into numerical representations called vectors.
 - **Vector Storage:** These vectors are stored in the vector store.
 - **Similarity Search:** When a new piece of data is presented, it's also converted into a vector, and the vector store is queried to find the most similar vectors.
-

KEY FEATURES:

- **High-dimensional data storage:** Handles data with numerous dimensions effectively.
 - **Similarity search:** Quickly finds the most similar vectors to a given query vector.
 - **Scalability:** Can handle large datasets and growing data volumes.
 - **Performance:** Optimized for efficient search and retrieval operations.
-

USE CASES:

- **Recommendation systems:** Recommending products, movies, or music based on user preferences or item similarities.
- **Search engines:** Improving search relevance by understanding the semantic meaning of queries.
- **Image and video search:** Finding similar images or videos based on visual content.
- **Chatbots and virtual assistants:** Understanding and responding to user queries in a natural language context.
- **Fraud detection:** Identifying patterns in fraudulent activities.

In essence, vector stores are crucial for applications that rely on semantic similarity and understanding the relationships between data points.

RESEARCH ON VECTOR STORE

As a part of this project, we have tried various vector stores namely, FAISS, CHROMADB, QUADRANT and PGVECTOR. Let us know about them

FAISS: A LIBRARY FOR EFFICIENT SIMILARITY SEARCH

FAISS (Facebook AI Similarity Search) is an open-source library developed by Meta AI specifically designed for efficient similarity search and clustering of dense vectors. It's particularly useful for handling large datasets where traditional search methods become inefficient(<https://ai.meta.com/tools/faiss/>).

KEY FEATURES:

- **Speed and efficiency:** FAISS is optimized for performance, capable of searching through billions of vectors quickly.
- **Diverse algorithms:** Offers a wide range of algorithms and data structures for different use cases and performance trade-offs.
- **GPU acceleration:** Leverages the power of GPUs (Graphics Processing Unit) to accelerate computations and improve search speed.

- **Python interface:** Provides a Python wrapper for easy integration into machine learning pipelines.

CHROMADB: AN AI-NATIVE VECTOR DATABASE

Chromadb is an open-source vector database designed specifically for AI applications. It excels at storing, managing, and retrieving vector embeddings, which are numerical representations of data used in machine learning.

KEY FEATURES:

- **Vector storage:** Efficiently stores and manages large-scale vector datasets.
- **Similarity search:** Performs fast nearest neighbor searches based on vector similarity.
- **Metadata support:** Allows you to associate metadata with vectors for filtering and querying.
- **Integration:** Works seamlessly with popular machine learning and natural language processing libraries.
- **Flexibility:** Supports various vector embedding models and distance metrics.

QDRANT

Qdrant is a vector database specifically designed for efficient similarity search. It's a powerful tool for handling high-dimensional data and is optimized for performance and scalability.

KEY FEATURES:

- **Vector similarity search:** Core functionality to find items like a given query vector.
- **High-dimensional data:** Efficiently handles large datasets with high-dimensional vectors.
- **Scalability:** Designed to handle growing datasets and increasing query loads.
- **Flexibility:** Supports various vector embedding models and distance metrics.

PGVECTOR: A POSTGRESQL EXTENSION FOR VECTOR SIMILARITY SEARCH

pgvector is a powerful open-source PostgreSQL extension that enables you to store, query, and index vectors directly within your database. It's designed to efficiently handle high-dimensional data, making it ideal for applications that rely on similarity search, such as:

- **Image search**
- **Recommendation systems**
- **Natural language processing (NLP)**
- **Anomaly detection**
- **Fraud detection**

KEY FEATURES:

- **Vector storage:** Stores high-dimensional vectors as a native data type.
- **Similarity search:** Performs efficient nearest neighbor searches based on various distance metrics (L2, cosine, inner product, etc.).
- **Indexing:** Supports both exact and approximate nearest neighbor search using indexes like HNSW and IVFFlat.

- **Integration:** Works seamlessly with other PostgreSQL features like transactions, joins, and indexing.

By combining the power of PostgreSQL with pgvector, you can build applications that leverage the benefits of both relational databases and vector databases. This allows you to store and query both structured and unstructured data within a single system, simplifying development and improving performance.

PGVECTOR VS. CHROMADB VS. FAISS VS. QDRANT

Disclaimer: The choice of vector database often depends on specific use cases, performance requirements, and integration needs. While PgVector offers several advantages, it's essential to evaluate all options based on our project's unique characteristics.

STRENGTHS OF PGVECTOR

- **SQL Integration:** Seamlessly integrates with PostgreSQL, leveraging its mature ecosystem and tooling.
- **Transactions:** Supports ACID transactions, ensuring data consistency and reliability.
- **Scalability:** Can scale horizontally using PostgreSQL's sharding capabilities.
- **Mature Ecosystem:** Benefits from PostgreSQL's extensive community and support.

STRENGTHS OF CHROMADB

- **Ease of Use:** User-friendly API and abstractions.
- **Flexibility:** Supports various vector stores and embedding models.
- **Cloud-Native:** Well-suited for cloud-based deployments.

STRENGTHS OF FAISS

- **Performance:** Optimized for efficient similarity search, particularly in high-dimensional spaces.
- **Flexibility:** Can be used as a standalone library or integrated into other systems.
- **Open-Source:** Active community and development.

STRENGTHS OF QDRANT

- **High-Performance Indexing:** Uses advanced techniques such as Hierarchical Navigable Small World and product Quantization to ensure low-latency vector retrieval.
- **Scalability:** It is designed to scale horizontally, which allows it to handle large volumes of data with query throughput.
- **Vector Quantization:** it supports vector quantization methods that reduce memory usage while maintaining high retrieval accuracy, which can be useful when working with large vector datasets.

- **Rich Feature Set:** Provides advanced features such as filtering, search by exact match, and support for complex query types making it versatile for various use cases beyond just similarity search.

PGVECTOR VS. COMPETITORS

- **SQL Integration:** PgVector's primary advantage lies in its tight integration with PostgreSQL. This allows for complex queries, joins, and transactions involving vector data, which is often challenging with other vector databases.
- **Performance:** While FAISS might offer raw performance advantages for certain workloads, PgVector's performance is often sufficient for many applications, especially when considering the benefits of SQL integration.
- **Maturity and Ecosystem:** PgVector benefits from PostgreSQL's long-standing maturity and extensive ecosystem, providing robust tooling and support.

In conclusion, PgVector is a compelling choice when SQL integration, transactional guarantees, and leveraging the PostgreSQL ecosystem are critical requirements. However, for specific use cases demanding extreme performance or flexibility, ChromaDB or FAISS might be worth considering.

Additional Factors:

- **Data Volume:** The size of our vector dataset will influence performance and storage costs.
- **Query Patterns:** The types of queries you'll perform (e.g., nearest neighbor, range search) can impact performance.
- **Deployment Environment:** Cloud-native vs. on-premises deployments might favor different options.

By carefully evaluating these factors and conducting benchmarks with our specific data, we have selected the pgvector vector database for our project.

For more information, refer the below GitHub URL :

https://github.com/deeplcit/tbc_application_1/blob/feature_Girish/research/llama3_pgvec.ipynb

DSPY: A FRAMEWORK FOR RAG EVALUATION

DSPY is a Python framework designed to facilitate the development and evaluation of Retrieval-Augmented Generation (RAG) systems. It provides a structured approach to building RAG pipelines, experimenting with different components, and measuring performance.

KEY FEATURES OF DSPY:

- **Unified Framework:** Offers a cohesive environment for constructing RAG systems, combining retrieval, generation, and evaluation components.
- **Flexibility:** Supports various retrieval systems, language models, and evaluation metrics.

- **Experimentation:** Enables rapid prototyping and experimentation with different RAG architectures.
- **Evaluation Metrics:** Provides built-in support for common RAG evaluation metrics.
- **Integration:** Can be integrated with popular libraries and tools for data processing, model training, and deployment.

HOW DSPY WORKS:

- **Define Components:** Specify the retrieval system, language model, and evaluation metrics.
- **Create Dataset:** Prepare a dataset containing documents and corresponding queries.
- **Build RAG Pipeline:** Construct the RAG pipeline using DSPy's building blocks.
- **Evaluate Performance:** Use DSPy's evaluation functions to assess the RAG system's performance.

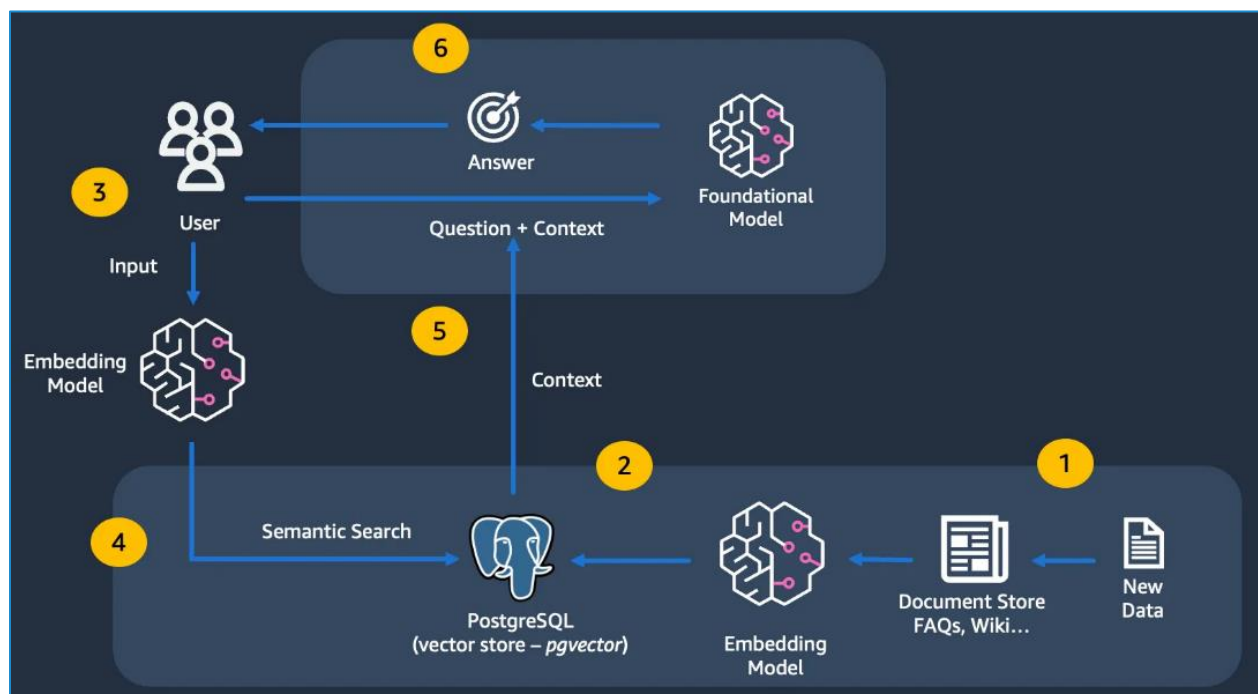
To know more about DsPy refer the below URLs:

https://github.com/deepcit/tbc_application_1/tree/feature_Deep/dspy_with_pgvector

https://github.com/deepcit/tbc_application_1/tree/feature_Deep/dspy_with_qdrant

ARCHITECTURE AND DESIGN

SYSTEM ARCHITECTURE



The above image depicts a Retrieval-Augmented Generation (RAG) architecture, which combines a large language model (LLM) with an information retrieval system to enhance the quality of the

LLM's responses. Here's a breakdown of the components and their functionalities based on the image:

User: This represents the person interacting with the system and initiating a query.

Question + Context: This signifies the user's input, which can include a question and any relevant contextual information that can help the system understand the user's intent better.

Embedding Model: This component takes the user's query and transforms it into a numerical representation, a vector, suitable for comparison with other data. It essentially captures the semantic meaning of the query.

Document Store: This is a repository that holds various documents or data sources that the RAG system can access to retrieve relevant information for the user's query. Examples can include FAQs, manuals, wikis, or news articles.

Semantic Search: This functionality employs the vector generated by the Embedding Model to search the Document Store for documents that share similar semantic meaning. In simpler terms, it searches for documents that are topically relevant to the user's query.

Foundational Model (LLM): This represents the large language model, which is a powerful learning system trained on massive amounts of text data. However, LLMs can sometimes lack access to specific or up-to-date information.

New Data (Optional): This indicates that the RAG system can incorporate real-time data streams or user interactions to continually update its knowledge and improve response accuracy.

Context (Embedding Model Output): This refers to the vector generated earlier by the Embedding Model, which captures the essence of the user's query. This contextual information helps the LLM understand how to best utilize the retrieved documents.

Answer: This is the final output generated by the LLM after considering the user's query, the retrieved documents from the document store, and the contextual information. The LLM leverages its knowledge and the retrieved data to formulate a comprehensive and informative response.

HERE ARE SOME OF THE KEY BENEFITS OF USING A RAG ARCHITECTURE:

Enhanced Accuracy: By incorporating real-world data from the document store, RAG helps LLMs generate more accurate and reliable responses compared to solely relying on their internal training data.

Improved Contextual Understanding: The use of contextual information from the user's query and retrieved documents allows the LLM to tailor its response to the specific situation.

Reduced Bias: RAG mitigates potential biases present in the LLM's training data by incorporating information from diverse external sources.

DESIGN DIAGRAMS

UML (Unified Modeling Language) is a standard visual modeling language used to design software systems. It provides a standardized way to visualize, specify, construct, and document the artifacts of a software system. We are using ERD diagram to describe our project artifacts.

ERD: ENTITY-RELATIONSHIP DIAGRAM

An Entity-Relationship Diagram (ERD) is a visual representation of how data is organized and related within a database system. It's a blueprint that outlines the entities (people, objects, or concepts), their attributes (properties or characteristics), and the relationships between them.

Key Components of an ERD:

- **Entities:** These represent real-world objects or concepts about which data is collected. For example, a customer, product, or order.
- **Attributes:** These are the properties or characteristics of an entity. For instance, a customer might have attributes like name, address, and phone number.
- **Relationships:** These define how entities are connected. For example, a customer can place multiple orders, indicating a one-to-many relationship between the customer and order entities.

Visual Representation: ERDs use specific symbols to represent these elements:

- **Entities:** Rectangles
- **Attributes:** Ovals or ellipses
- **Relationships:** Diamonds or lines with arrowheads

By understanding the entities, attributes, and relationships within a system, ERDs help database designers create efficient and effective databases. They are widely used in software engineering, database design, and business information systems.

CHATFLIX DATA: ENTITY-RELATIONSHIP DIAGRAM

Below ERD diagram contains the details of the tables and their relationships used as a part of this project

admin_contact	Character varying(1000)	Contact information for the admin
admin_pic	Character varying(1000)	Path to the admin's profile picture
admin_location	Character varying(1000)	Location of the admin
created_at	Timestamp without time zone	Date and time when the admin record was created

KNOWLEDGE BASE

Description: Contains details about knowledge resources available to users.

Usage:

- Storing and managing various types of files added.
- Providing users with access to relevant information.

TABLE STRUCTURE & DESCRIPTION:

Column Name	Type	Description
kb_id	Serial	Unique identifier for the knowledge base
kb_title	Character varying(1000)	Title of the knowledge base
kb_type	Character varying(1000)	Type of knowledge base (e.g., article, tutorial, FAQ)
kb_status	Character varying(1000)	Status of the knowledge base (e.g., published, draft)
kb_desc	Character varying(1000)	Description of the knowledge base
kb_created_at	Timestamp without time zone	Date and time when the knowledge base was created
kb_file	Character varying(1000)	Path to the knowledge base file (if applicable)

USERS_TBL

Description: Contains information about system users.

Usage:

- User authentication and management.
- Tracking user activity and preferences.
- Providing personalized content or recommendations.

TABLE STRUCTURE & DESCRIPTION:

Column Name	Type	Description
user_id	Serial	Unique identifier for the user
user_name	Character varying(1000)	Name of the user
user_email	Character varying(1000)	Email address of the user
user_username	Character varying(1000)	Username for the user
user_password	Character varying(1000)	Password for the user
user_contact	Character varying(1000)	Contact information for the user
user_pic	Character varying(1000)	Path to the user's profile picture
user_location	Character varying(1000)	Location of the user
user_created_at	Timestamp without time zone	Date and time when the user record was created

LANGCHAIN_PG_COLLECTION

Description: Represents collections of embeddings used for semantic search or similar applications.

Usage:

- Organizing embeddings based on specific topics or domains.
- Efficiently retrieving relevant embeddings for query processing.

TABLE STRUCTURE & DESCRIPTION:

Column Name	Type	Description
collection_id	UUID	Unique identifier for the collection
name	Character varying(1000)	Name of the collection
metadata	JSON	Additional metadata about the collection
uuid	UUID	unique identifier for the embeddings

LANGCHAIN_PG_EMBEDDING

Description: Stores vector representations of documents for semantic search or other machine learning tasks.

Usage:

- Enabling similarity search based on semantic meaning.
- Powering recommendation systems or information retrieval.

TABLE STRUCTURE & DESCRIPTION:

Column Name	Type	Description
embedding_vector	(Data type unspecified)	Vector representation of the document
document	Character varying(1000)	Text content of the document
cmetadata	JSON	Additional metadata about the embedding
custom_id	Character varying(1000)	Custom identifier for the embedding
uuid	UUID	Unique identifier for the embedding

Relationship:

- **admin_tbl** and **knowledge_base** have a one-to-many relationship, with an admin being able to create multiple knowledge bases.
- **langchain_pg_collection** and **langchain_pg_embedding** have a one-to-many relationship, with a collection containing multiple embeddings.
- **users_tbl** and **knowledge_base** have a many-to-many relationship, with users being able to access multiple knowledge bases and knowledge bases being accessible to multiple users.

To know more about the refer the table details refer the below URL:

https://github.com/deeplcit/tbc_application_1/tree/main/database

TECHNOLOGY STACK

We have used various technical stack as a part of this project:

Data Preprocessing: Python

Data Storage: Postgres Database

Modelling: Langchain, Ollama

Backend Development: Python Flask API, Java Script, PHP

Frontend Development: HTML, JavaScript, CSS, Bootstrap

FEATURES AND FUNCTIONALITIES

CORE FEATURES

The chatbot leverages the college's website as its knowledge base to answer student questions and provide information across various departments. Here is a breakdown of its key features categorized for easy access (for a more comprehensive list, see the Core Features Document [<https://rise.articulate.com/share/dggfjq-kOdqZh5GtxawG7XPBLP20wsEB#/lessons/mX2c79JikKquBDh63BV8BjVQBZDqMhD5>]).

Awareness & Safety

- Scam & Prevention
- Human Rights 101
- Sexual Assault Policy

Student Success

- Tutoring Services
- Accessibility & Library

College Services

- Transit
- Dependent Children
- Student ID Card
- Activities & Events
- Hotel Reservations
- Update Information

Front Desk Services

- First Aid Kits
- AED
- Lost & Found

Clubs

- Arts Club
- Games Club
- Dance & Music Club

Get Involved

- LCIT Volunteers
- Student Ambassadors

Community Services

- LCIT Discounts
- Food
- Museum
- Volunteering Opportunities

Policies & Procedures

- TBC Policies
- TBC Procedures

Along with the above data sources we have added the data from the below mentioned URL and all the sub-URLs from them by converting them into PDFs:

<https://loyalistcollege.com/about-loyalist/policies/>

INTEGRATION AND APIS

WHAT IS AN API?

API stands for Application Programming Interface. It's essentially a set of rules or protocols that allows different software applications to communicate and interact with each other. Think of it as a middleman that facilitates the exchange of data and services between two systems.

HOW DOES AN API WORK?

1. **Request:** An application sends a request to the API with specific instructions or data.
2. **Processing:** The API processes the request, often interacting with a database or other systems to retrieve or manipulate data.
3. **Response:** The API sends a response back to the original application with the requested information or the result of the action.

WHY ARE APIS IMPORTANT?

Efficiency: APIs allow developers to reuse existing functionalities, saving time and effort.

Innovation: APIs enable the creation of new applications and services by combining data and functionalities from different sources.

Integration: APIs facilitate seamless integration between different software systems.

Scalability: APIs can handle increased workloads and traffic, ensuring the system can grow as needed.

EXAMPLES OF APIS IN ACTION:

Weather apps: Fetching weather data from a weather service's API.

Social media sharing: Sharing content on various platforms using their respective APIs.

Payment gateways: Processing payments through payment service provider APIs.

Mapping services: Displaying maps and directions using mapping APIs.

In essence, APIs are the building blocks of modern software development, enabling developers to create interconnected and dynamic applications.

API OPERATIONS: THE BUILDING BLOCKS OF INTERACTIONS

API operations define the actions that can be performed on a specific resource through an API. These operations are typically mapped to HTTP methods for standardized communication.

COMMON API OPERATIONS

- **GET:** Retrieves data or resources.
 - Example: Fetching a list of users, getting details of a specific product.
- **POST:** Creates new resources or data.
 - Example: Adding a new user, placing an order.
- **PUT:** Updates an entire resource.
 - Example: Modifying all details of a user profile.
- **PATCH:** Updates specific fields of a resource.
 - Example: Changing a user's password or email address.
- **DELETE:** Removes a resource.
 - Example: Deleting a user account, canceling an order.

ADDITIONAL OPERATIONS (LESS COMMON)

- **HEAD:** Retrieves metadata about a resource without the actual content.
- **OPTIONS:** Provides information about the communication options available on the resource.
- **TRACE:** Echoes the request back to the client.

EXAMPLE: A USER RESOURCE

For a user resource, common operations might include:

- **GET /users:** Retrieve a list of users
- **GET /users/{id}:** Retrieve details of a specific user
- **POST /users:** Create a new user
- **PUT /users/{id}:** Update all details of a user
- **PATCH /users/{id}:** Update specific fields of a user
- **DELETE /users/{id}:** Delete a user

BEST PRACTICES FOR API OPERATIONS

- **Adhere to HTTP standards:** Use appropriate HTTP methods for actions.

- **Consistent naming:** Use clear and consistent naming conventions for resources and operations.
- **Error handling:** Provide informative error messages and status codes.
- **Versioning:** Implement versioning for API changes.
- **Security:** Protect your API with authentication and authorization mechanisms.
- **Documentation:** Provide clear and comprehensive documentation for API consumers.

API DEVELOPMENT IN PYTHON

There are various API implementations in the market and to develop custom APIs for our project we have chosen the Python language as most of our modelling code is done in Python language.

There are numerous API tools available in python. Let us discuss the most famous ones.

FASTAPI

FastAPI is a modern, high-performance Python web framework designed for building APIs. It leverages advanced Python features like type hints and asynchronous programming to deliver exceptional speed and efficiency. FastAPI emphasizes developer experience with features like automatic data validation. It is well-suited for building robust and scalable APIs.

DJANGO

Django is a full-featured Python web framework that follows the Model-View-Controller (MVC) architectural pattern. It provides a comprehensive set of tools for building complex web applications, including an ORM for database interactions, a templating engine, and a built-in admin interface. Django prioritizes rapid development and security, making it a popular choice for large-scale projects.

FLASK

Flask is a lightweight, flexible Python web framework. It provides essential tools for building web applications and APIs, without imposing strict structure. This simplicity makes Flask easy to learn and use for small to medium-sized projects. Developers have full control over the components used, allowing for customization and optimization.

FLASK API VS FASTAPI VS DJANGO

Let us compare the API tools in python

Feature	Flask	FastAPI	Django
Philosophy	Minimalistic	Fast and efficient	Full featured
Best for	Small projects, APIs, microservices	APIs, real-time apps, microservices	Complex web apps, CMS
Performance	Medium	High	Medium

Complexity	Low	Low	High
------------	-----	-----	------

FLASK: IDEAL CHOICE FOR OUR CHATBOT PROTOTYPE

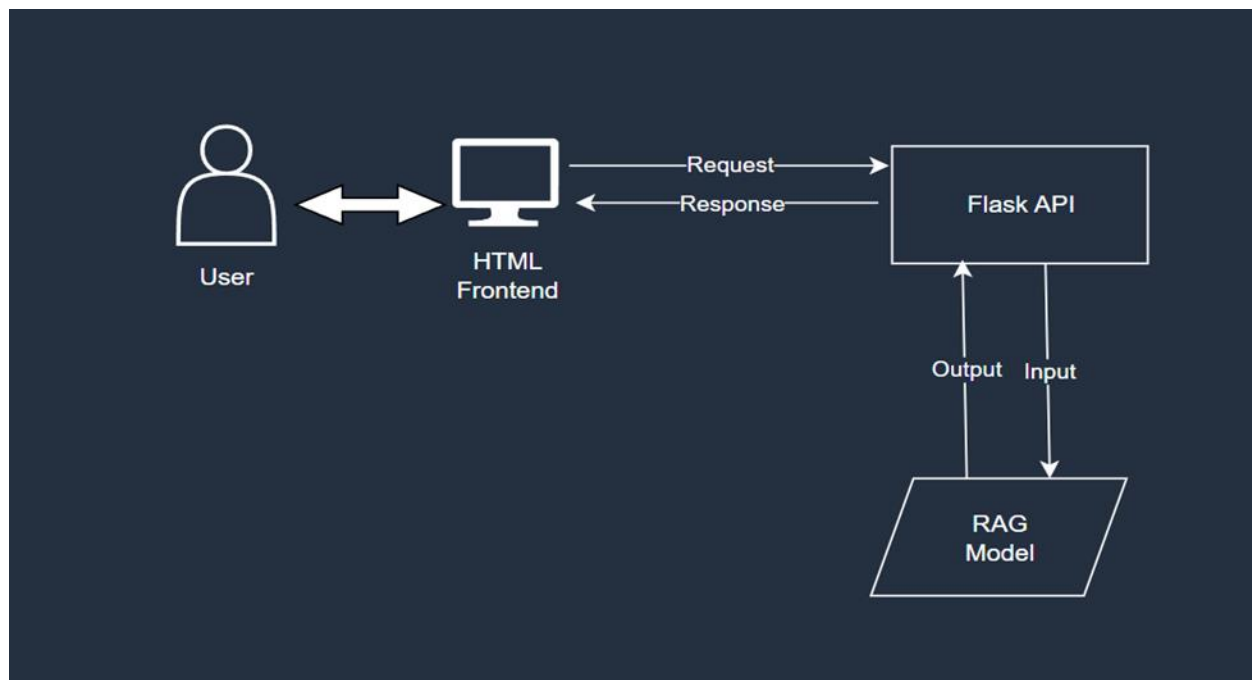
For our chatbot prototype, Flask emerges as the optimal framework due to its exceptional suitability for rapid development, flexibility, and scalability.

- **Rapid Development:** Flask's minimalist approach allows for swift prototyping, enabling us to quickly iterate on the chatbot's functionality and user interface. Its straightforward structure minimizes development time and accelerates the creation of a functional prototype.
- **Flexibility:** As a chatbot prototype, requirements may evolve rapidly. Flask's adaptability allows for easy modifications and integrations, ensuring the project can accommodate changing needs without significant overhauls.
- **Scalability:** While Flask is primarily designed for smaller-scale applications, its modular architecture provides a foundation for future expansion. As the chatbot matures, we can incrementally incorporate additional features and handle increased user loads.

By leveraging Flask's strengths, we can efficiently build a robust chatbot prototype that serves as a solid foundation for future development.

INTEGRATION POINTS

In this section we will discuss how APIs are integrating all our systems and the illustrations below describe the same.



The Flask API serves as an interface to handle user queries using a Retrieval-Augmented Generation (RAG) model. Below is the breakdown of the above illustration:

1. **User Interaction:** The diagram initiates with a user interacting with an HTML frontend. This frontend could be a web application, a mobile app, or any interface that allows user input.
2. **Request to Flask API:** The user's input or query is sent as a request to the Flask API. This Flask API acts as an intermediary, handling the communication between the user and the RAG model.
3. **Flask API and RAG Model Interaction:** The Flask API receives the user's input and forwards it to the RAG model. RAG (Retrieval Augmented Generation) model is responsible for processing the user's query and retrieving relevant information from an external knowledge base or database. The RAG model then generates a comprehensive and informative response based on the retrieved information.
4. **Response to User:** The Flask API receives the generated response from the RAG model. The Flask API then sends this response back to the user through the HTML front-end. The user ultimately receives the final output, which is the result of the RAG model's processing and the Flask API's mediation.

In essence, the diagram illustrates a system where a user interacts with an HTML frontend, sending a request to a Flask API. The Flask API then interacts with a RAG model to process the user's query and generate a response, which is returned to the user.

API TESTING TOOLS: POST MAN

WHAT IS POSTMAN?

Postman is a popular API development environment that helps developers to design, build, test, and document APIs. It provides a user-friendly interface for interacting with APIs and managing API requests and responses

WHY USE POSTMAN?

API Testing:

- Execute test suites and collections of API requests.
- Validate API responses against expected results.
- Identify and troubleshoot API issues.

Postman simplifies the API development process by providing a centralized platform for managing API-related tasks. It helps improve API quality, efficiency, and collaboration among development teams.

APIS DEVELOPED IN PROJECT

List of APIs used, including endpoints, request/response formats, and authentication methods. This is the integration of the RAG chatbot within the Flask API, detailing the architecture, components, and workflow.

Description: The Flask application acts as the intermediary between the frontend and the backend services. It exposes several endpoints to handle user queries and manage interactions with the chatbot.

1. 'POST /ai': Receives user queries and returns the chatbot's response.
 - a. Request Format: JSON
 - i. Ex. {"message": "Number of courses available in Loyalist college?"}
 - b. Response Format: String
 - i. Ex. {"response": "There are n number of courses available."}
2. 'POST /pdf': Converts the pdf document uploaded by user into embeddings and store the embeddings into PostgreSQL database.
 - a. Request Format: PDF (document)
 - b. Response Format: String (Status)
 - i. Ex. {"Successfully Uploaded"}, {"No file part"}, etc...}

Key Functions:

- **Query Processing:** Parses and preprocesses user queries.
- **Retrieval:** Utilizes pgvector to fetch relevant documents from the database.
- **Augmentation:** Combines the retrieved information with the user's query.
- **Generation:** Uses the RAG model to generate a coherent and contextually relevant response.

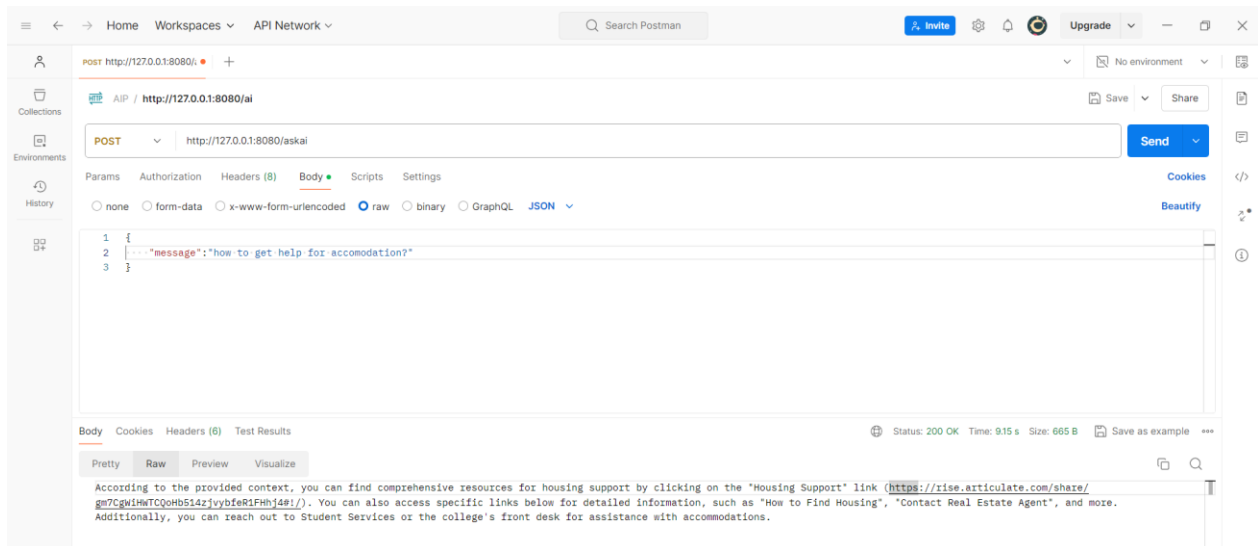
Integration Workflow:

1. **User Query Submission:** The user submits a query through the front end.
2. **API Request Handling:** The query is sent to the POST /chat endpoint of the Flask API.
3. **Query Processing:** The business logic layer processes the query and converts it into a suitable format for retrieval.
4. **Document Retrieval:** The system uses pgvector to find the most relevant documents from the database based on the vector similarity.
5. **Augmentation:** The retrieved documents are combined with the user's query to form an augmented query.
6. **Response Generation:** The RAG model generates a response based on the augmented query.
7. **API Response:** The generated response is sent back to the frontend through the Flask API.
8. **Response Display:** The front end displays the chatbot's response to the user.

To know about the API's used in project refer the below mentioned URL:

https://github.com/deeplcit/tbc_application_1/tree/main/API

Working screenshot of the API on Postman:



DATA MANAGEMENT

POSTGRESQL DATA STORAGE AND SECURITY ON LOCAL MACHINE

Since the data is confidential college data we are storing the data in PostgreSQL on a local machine, security is paramount. Here's a breakdown of the requested information:

DATA STORAGE

- **How:** PostgreSQL stores data in tables. Each table consists of rows (records) and columns (fields). Data is physically stored on our local machine's disk drive, likely formatted according to our operating system (e.g., ext4 for Linux, NTFS for Windows).
- **Where:** The exact location depends on our operating system and PostgreSQL configuration. By default, PostgreSQL data directories are typically found in:
 - ❖ Linux/macOS: `/var/lib/postgresql/<version>/data` (replace `<version>` with our installed PostgreSQL version)
 - ❖ Windows: `C:\Program Files\PostgreSQL\<version>\data`

DATA PRIVACY

Securing our local PostgreSQL instance is crucial for data privacy. Here are some measures to consider:

Data Encryption:

- We can implement data encryption at rest to protect stored data even if someone gains access to our disk. PostgreSQL supports encryption using tools like pgcrypto.

- Consider encrypting backups as well for added security.

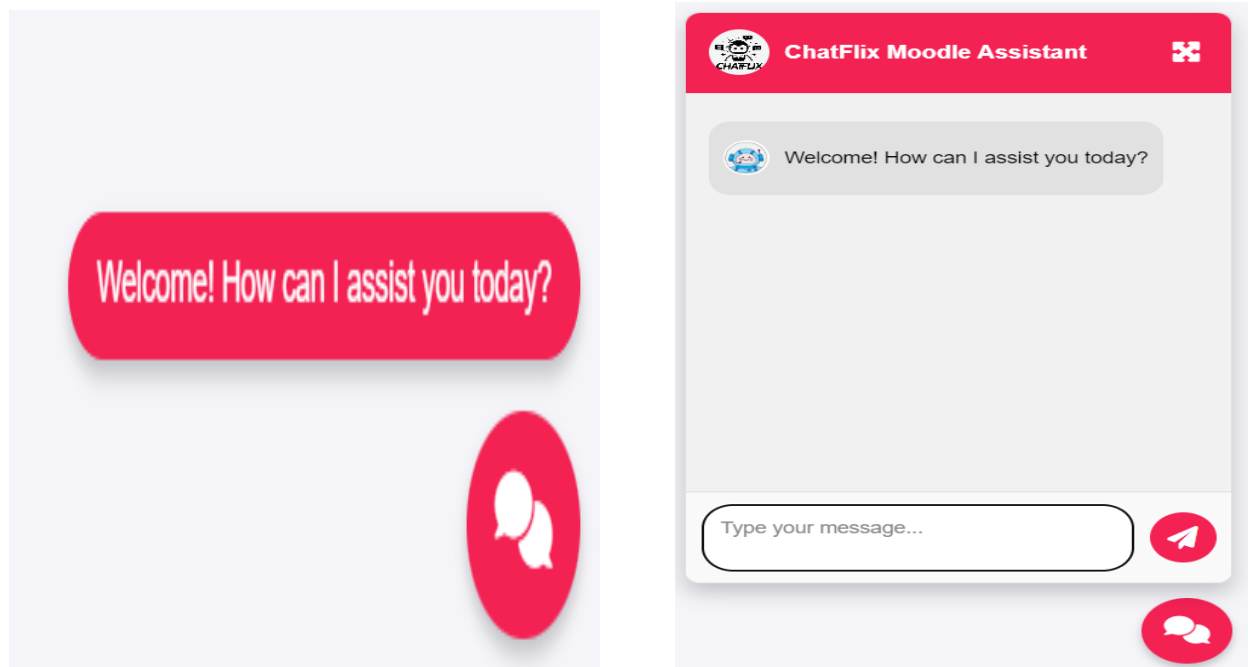
DATA BACKUP AND RECOVERY

Regular Backups:

- We can establish a backup schedule to create copies of our database at regular intervals (e.g., daily, weekly). but the changes in data are not frequent we can load the data in adhoc manner whenever necessary
- As we are using Postgres, we can consider using tools like pg_dump or third-party backup solutions for automated backups.

USER INTERFACE (UI)

CHAT INTERFACE



The LCIT chatbot interface will be a separate window that appears on the bottom right corner of the college website homepage. It will feature a friendly avatar and a text input field where users can type their questions. Additionally, the interface will have a button for maximizing the chat window. To know more about the code explanation. Please, refer to the readme file from the below repo.

GitHub: https://github.com/deeplcit/tbc_application_1/tree/main/moodle

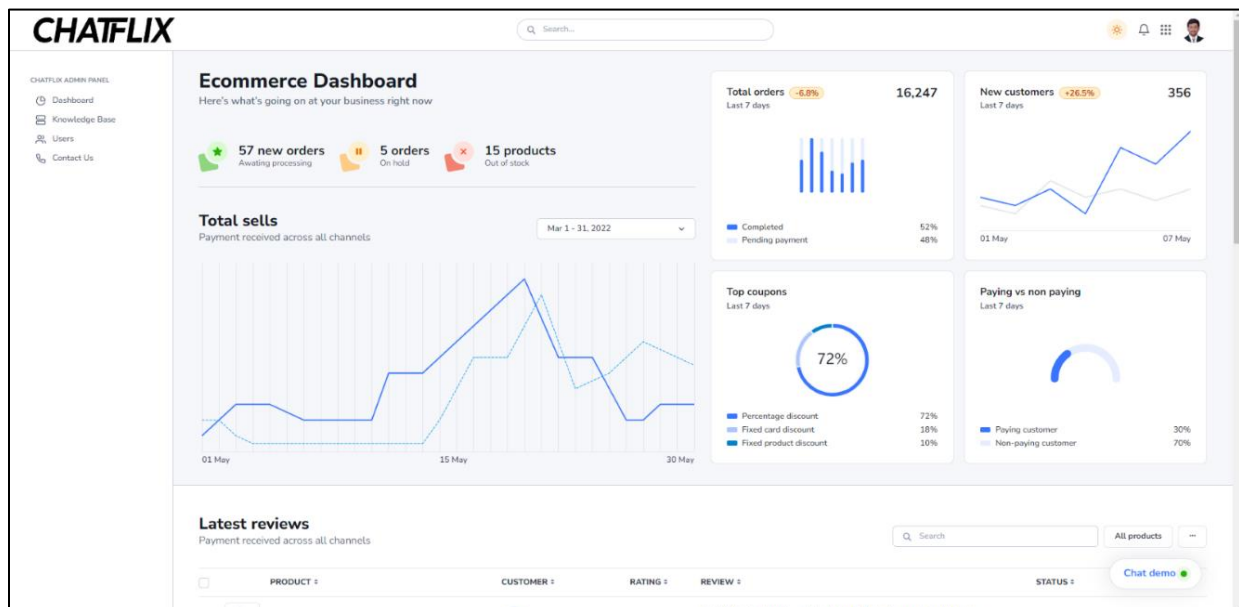
ADMIN PANEL

The admin panel is a PHP-based interface designed to manage the chatbot project, which utilizes a Flask API with RAG (Retrieval-Augmented Generation) and pgvector. The panel provides functionalities to manage PDFs, view admin information, and monitor the dashboard. Below is a detailed description of each tab and its functionality.

Tabs and Their Functions:

1. Dashboard

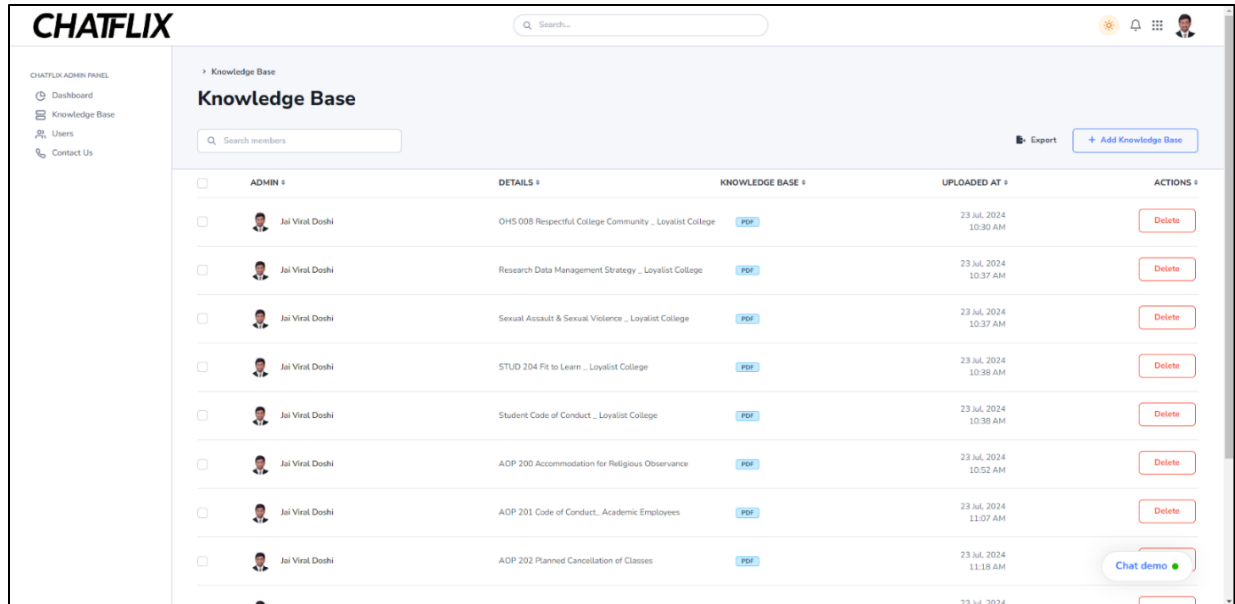
Function: The Dashboard tab provides an overview of the chatbot's current status and key metrics.



2. Knowledge Base

Function: The Knowledge Base tab allows the admin to manage the PDFs that are used to train or update the chatbot's knowledge. This includes:

- **Viewing PDFs:** A list of all uploaded PDFs.
- **Uploading PDFs:** An interface to upload new PDF files to be added to the chatbot's knowledge base.
- **Deleting PDFs:** Options to remove outdated or unnecessary PDF files from the knowledge base.



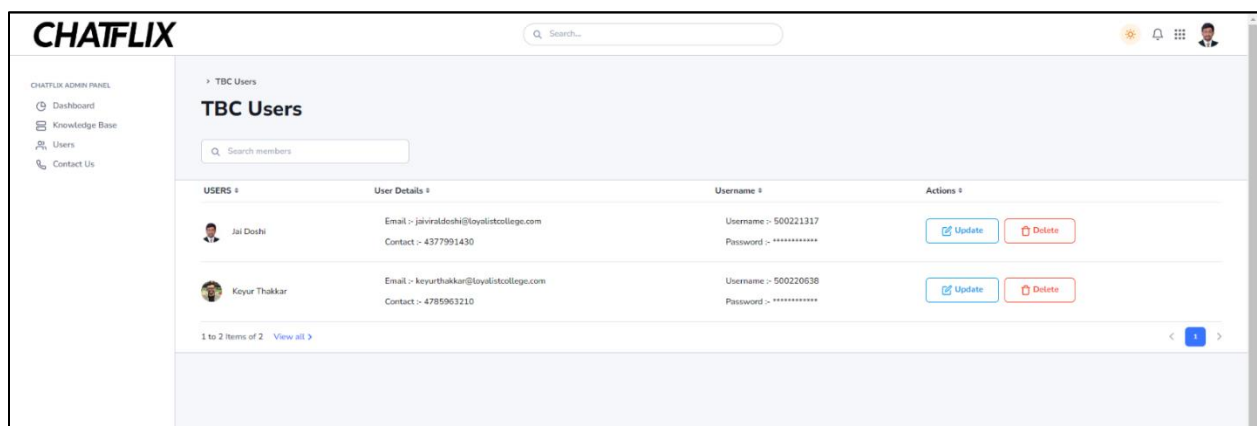
How It Works:

- **File Handling:** Uploaded files are saved to a server directory and their paths are stored in the database.
- **Update Process:** The Flask API is notified of the new PDFs, which are then indexed for the RAG model using pgvector to update the chatbot's knowledge.

3. Admin Information

Function: The Admin Information tab provides details about the users who have access to the admin panel. This includes:

- **User List:** Display of all admin users.
- **Username:** Information on usernames of the user.
- **User Details:** Provides details such as email and contact information.



How It Works:

- **User Management:** Admin details are retrieved from the database and displayed in a user-friendly format.
- **Information Updates:** Changes in user information are handled through PHP scripts that update the database accordingly.

Implementation Details:

Database Integration:

- **PDF Management:** Tables to store metadata about uploaded PDFs, including file paths and upload timestamps.

Admin Management: Tables to manage admin user information and their permissions.

Flask API Communication:

- **Endpoints:** API endpoints that allow the admin panel to interact with the Flask API, such as updating the chatbot's knowledge base or fetching system statistics.

To know more about the admin panel refer the following URL:

https://github.com/deepicit/tbc_application_1/tree/main/Chatflix/admin

SECURITY

AUTHENTICATION AND AUTHORIZATION

Authentication ensures that only authorized users can access the admin panel. This process involves verifying the identity of users attempting to log in.

ChatFlix Authentication

Give yourself some hassle-free development process with the uniqueness of Phoenix!

Fast
Simple
Responsive

CHATFLIX

Sign In

Get access to your account

[Sign in with google](#)

[Sign in with facebook](#)

or use email

USERNAME

PASSWORD

☒ Remember me [Forgot Password?](#)

[Sign In](#)

[Create an account](#)

HOW USERS ARE AUTHENTICATED

1. Login Page:

- Users are presented with a login page where they must enter their credentials (username and password).

2. Credential Verification:

- Upon submission, the credentials are sent to the server where they are verified against the stored user data in the database.
- Passwords are hashed using a secure hashing algorithm (e.g., bcrypt) before being stored in the database. During login, the entered password is hashed and compared with the stored hash.

3. Session Management:

- If the credentials are correct, a session is created for the user. This session typically involves generating a session ID, which is stored in the user's browser as a cookie.
- The session ID is also stored on the server to track the authenticated session.

TESTING AND QUALITY ASSURANCE

TESTING STRATEGIES:

Types of testing performed (e.g., unit tests, integration tests, user acceptance tests).

API:

These are the steps taken to test the Flask API, using Postman. The goal is to ensure the API endpoints are working as expected and to validate the chatbot's responses.

PREREQUISITES:

Postman Installed: Ensure you have Postman installed on our machine. You can download it from Postman's official website.

Running Flask API: The Flask API should be running locally or on a server. Note the base URL of our API (e.g., <http://localhost:5000>).

STEPS TO TEST THE API:

SETTING UP POSTMAN:

1. **Open Postman:** Launch the Postman application.
2. **Create a New Collection:** Click on "New Collection" and give it a name.

TESTING ENDPOINTS:

Flask API has the following endpoint; we will go through one as example:

'POST /api/ai'

ENDPOINT 1: 'POST/API/AI'

Create a New Request: Click on "New" and select "Request". Name it "Response Check" and save it to the "RAG Chatbot API" collection.

1. **Set Request Type:** Set the request type to 'POST'.
2. **Enter URL:** Enter the URL for the health check endpoint, e.g., 'http://localhost:5000/api/health'.
3. **Send Headers:** In the "Headers" tab, add a new key-value pair:
Content-Type : 'application/json'
4. **Set Body:** In the "Body" tab, select "raw" and set the following JSON payload:
{"query": "What is the college's policy on late submissions?"}
5. **Send Request:** Click "Send".
6. **Verify Response:** Check the response status code and body. A typical response might be:
{"response": "The college policy on late submissions is as follows..."}

VERIFYING RESPONSES:

1. **Status Codes:** Ensure all endpoints return appropriate HTTP status codes (e.g., 200 for successful requests).
2. **Response Bodies:** Verify the response bodies contain the expected data. Check for correct keys, values, and formats.
3. **Error Handling:** Test for common errors (e.g., missing fields in the request body) to ensure the API handles them gracefully.

To know more about the API used in project refer the following URL:

https://github.com/deeplicit/tbc_application_1/tree/main/API

METRICS IN PERFORMANCE EVALUATIONS

EVALUATING RAG MODEL PERFORMANCE

Evaluating the performance of a Retrieval-Augmented Generation (RAG) model is a complex task due to the interplay between the retrieval component and the language model. Here are key areas to focus on:

RETRIEVAL COMPONENT EVALUATION

- **Precision and Recall:** Assess the ability of the retrieval system to find relevant documents for a given query.
- **Diversity:** Evaluate the system's ability to retrieve a diverse set of documents.
- **Speed:** Measure the time taken to retrieve relevant documents.

GENERATION COMPONENT EVALUATION

- **Factuality:** Assess the accuracy of the generated text based on the retrieved documents.
- **Relevance:** Evaluate how well the generated text addresses the user's query.
- **Coherence:** Assess the overall coherence and fluency of the generated text.
- **Consistency:** Ensure that the generated text aligns with the information presented in the retrieved documents.

END-TO-END EVALUATION

- **Human Evaluation:** Conduct user studies to gather feedback on the overall quality of the RAG system.
- **Metric-based Evaluation:** Employ metrics like BLEU, ROUGE, or METEOR to compare generated text with human-written references.
- **Task-specific Evaluation:** Assess the performance of the RAG system on specific tasks or domains.

OPTED EVALUATION STRATEGY

We have used End-to-End Evaluation strategy for evaluation of our RAG model and below are the details:

To compare the performance of RAG models made using different large language model (LLM's) two metrics were used:

ROUGE SCORE

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) is a set of metrics and a software package designed to evaluate text summaries generated using AI. Here we have used the rouge score to compare the AI generated answers with the ground truth answer.

HOW IT WORKS:

- **Comparison:** ROUGE compares an automatically produced summary or translation (the "candidate") against a reference summary or translation (human-produced).
- **N-grams:** It focuses on n-grams (sequences of n words) that overlap between the candidate and reference summaries.
- **Metrics:** Several ROUGE metrics exist, including:
- **ROUGE-N:** Measures the overlap of n-grams (unigrams, bigrams, trigrams, etc.) between the candidate and reference summaries.

INTERPRETATION:

- **A higher ROUGE score indicates better similarity** between the candidate and reference summaries. A perfect match would result in a ROUGE score of 1.
- ROUGE scores are often used with other metrics for a comprehensive evaluation.

LIMITATIONS:

- ROUGE primarily focuses on n-gram overlap, which might not capture semantic similarity effectively.
- It does not consider the ordering of words within sentences.

```
from rouge import Rouge

rouge = Rouge()
score = rouge.get_scores(generated_answer, answer)[0]
```

RESPONSE RERANKING

Using a prompt, we can ask AI to compare the generated answer and the ground truth answer and provide a rating between 1 to 5 to determine the similarity between the answers using DsPy.

Refer the below GitHub URL for our metrics evaluation

https://github.com/deeplcit/tbc_application_1/blob/feature_Deep/5_metrics.ipynb

HUMAN EVALUATION FOR RAG MODEL ASSESSMENT

While automatic metrics like ROUGE, BLEU, and METEOR provide quantitative assessments of RAG model performance, human evaluation offers a more nuanced and comprehensive understanding of the model's capabilities. This section outlines the methodology and considerations for conducting human evaluations to assess RAG system quality.

We have asked our project coordinator and project advisor to test our model and evaluate the RAG model that we have developed.

DEVELOPER GUIDE

DATA LOADING AND MODELLING

Below is the list of guidelines that need to be followed as a part of Data Loading and Modelling

WHAT IS OLLAMA?

Ollama is an open-source platform designed to simplify running large language models (LLMs) like me on our local machine. Here is a breakdown of what it offers:

Local LLM Playground: Ollama allows you to download, install, and run various open-source LLMs directly on our computer. This eliminates the need to rely on cloud-based LLM services or complex infrastructure setups.

Model Management: Ollama uses a concept called a "Modelfile" which packages everything needed to run an LLM – the model weights, configuration details, and data – into a single file. This simplifies model management and makes it easy to switch between different LLMs.

Customization Options: Ollama offers some level of customization for advanced users. You can fine-tune LLM parameters, adjust settings, and potentially tailor the models' behavior to our specific needs and preferences.

BENEFITS OF USING OLLAMA:

Accessibility: Ollama makes powerful LLMs more accessible to a wider range of users, from researchers and developers to enthusiasts who want to experiment with these models locally.

Control: Running LLMs locally gives you more control over our data and how the models are used.

Reduced Costs: Compared to cloud-based LLM services, using Ollama can potentially reduce costs associated with LLM usage, especially for frequent interactions.

THINGS TO CONSIDER:

Hardware Requirements: Running LLMs can be resource intensive. Ollama might require a computer with a powerful graphics processing unit (GPU) for optimal performance.

Technical Expertise: While Ollama aims to be user-friendly, some technical knowledge might be helpful for setup, configuration, and troubleshooting.

Limited Model Selection: Ollama primarily focuses on open-source LLMs. You might not find the latest or most advanced commercial models available through this platform.

OLLAMA INSTALLATION

Ollama offers installation options for Windows, macOS, and Linux. Here's a breakdown of the steps for each operating system:

Windows:

1. **Download:** Head over to the official Ollama website: <https://ollama.com/>
2. **Installation:** Locate the download button and choose the Windows executable file (.exe). Once downloaded, simply run the executable file. Ollama will handle the installation process automatically.

macOS:

1. **Download:** Visit the Ollama website as mentioned above.
2. **Installation:** Download the macOS package file (.dmg). Open the downloaded file and drag the Ollama application icon into our Applications folder.

Linux:

1. **Terminal Access:** Open our terminal application.
2. **Installation Command:** Execute the following command in our terminal:
`curl -sL https://raw.githubusercontent.com/openai/ollama/master/install.sh | bash -s -- -b`

Explanation of the Command:

- `curl`: This tool retrieves data from the internet.
- `-sL`: These flags instruct `curl` to operate silently and follow redirects.
- <https://raw.githubusercontent.com/openai/ollama/master/install.sh>: This specifies the URL of the Ollama installation script.
- `bash -s -- -b`: This pipes the downloaded script into the bash shell for execution with the `-b` flag. The `-b` flag indicates a non-interactive installation.

Once the installation is complete, you can launch Ollama using the following commands depending on our OS:

- **Windows:** Open the Ollama application icon from our Start menu or desktop shortcut (if created).
- **macOS:** Open the Ollama application from our Applications folder.
- **Linux:** Run the command `ollama serve` in our terminal. This starts the Ollama server.

Additional Notes:

- You might need administrator privileges for installation on Windows and macOS.
- For Linux, ensure you have `curl` installed before running the installation command. You can typically install `curl` using our package manager (e.g., `sudo apt install curl` for Ubuntu/Debian).
- After installation, you can explore Ollama commands and functionalities through the terminal using the `ollama` prefix (e.g., `ollama list` to view available models).

PULLING THE MODELS FROM OLLAMA

Ollama offers various open-source models to use in our local operating system and as a part of this project we have opted for the Llama3 model, an open-source model given by META.

PULLING AND RUNNING LLAMA3 WITH OLLAMA ON DIFFERENT OPERATING SYSTEMS

Here is a detailed guide on pulling and running the Llama3 model with Ollama across various operating systems:

Prerequisites:

- **Ollama Installation:** Ensure you have Ollama installed on our system. Refer to the previous explanation for installation steps specific to our OS (Windows, macOS, or Linux).
- **Administrator Privileges:** You might need administrative access for some steps depending on our OS.

Steps:

1. **Open Terminal (Linux/macOS) or Command Prompt (Windows):** Locate and launch our terminal or command prompt application.
2. **Pull the Llama3 Model:** Use the following command to download the Llama3 model: `ollama pull llama3`

Explanation:

- `ollama`: This invokes the Ollama command-line interface.
- `pull`: This tells Ollama you want to download a model.
- `llama3`: This specifies the name of the model you want to pull (Llama3 in this case).

Download Size: Be aware, the Llama3 model is quite large (around 39GB for the quantized version). Download time will depend on our internet speed.

Running Llama3 (All Operating Systems):

Once the download completes, you can start the Llama3 model using this command: `ollama run llama3`

Explanation:

- `ollama`: Invokes the Ollama command-line interface.
- `run`: This instructs Ollama to launch the specified model.
- `llama3`: This specifies the name of the model you want to run.

Ollama Interface: After running the command, Ollama will initiate the model and potentially display some information about it. You should now be able to interact with the Llama3 model by entering text prompts in our terminal.

1. **Interact with Llama3:** Type our prompts or questions and press Enter to send them to the model. Llama3 will then generate a response based on our input.
2. **Exit Ollama (All Operating Systems):** To exit the Ollama session and stop the Llama3 model, press Ctrl+C (Windows/Linux) or Command+C (macOS) in our terminal.

ADDITIONAL NOTES:

- **Model Selection:** You can list available models using the command `ollama list`. This can help you explore other models supported by Ollama.
- **Advanced Usage:** For more advanced functionalities like model configuration and customization, refer to the Ollama documentation: <https://ollama.com/>

TROUBLESHOOTING:

- **Hardware Limitations:** Running large models like Llama3 might require a computer with a powerful GPU. If you encounter performance issues, consider using the smaller 4-bit quantized version of Llama3 (download with `ollama pull llama3:4b`).
- **Technical Issues:** If you face installation or execution problems, consult the Ollama documentation or online forums for troubleshooting solutions.

By following these steps, you should be able to successfully pull and run the Llama3 model using Ollama on our computer!

POSTGRESQL AND PGVECTOR INSTALLATION

INSTALLING POSTGRESQL AND PGVECTOR ON DIFFERENT OPERATING SYSTEMS

Here's a breakdown of installing PostgreSQL and the pgvector extension on various operating systems:

Common Considerations:

- **Package Managers:** Most operating systems utilize package managers for software installation. These instructions leverage those tools.
- **PostgreSQL Version:** The specific commands might vary slightly depending on the desired PostgreSQL version.
- **User with Privileges:** You might need to use an account with administrative privileges to install software.

Linux Distributions (Ubuntu, Debian, etc.):

1. **Update Package Lists:** Ensure you have the latest package information:
`sudo apt update`
2. **Install PostgreSQL:** Install the main PostgreSQL package and the pgvector extension in one step:
`sudo apt install postgresql-15-pgvector`
Replace 15 with our desired version
3. **Initialize PostgreSQL:** After installation, run the following command to initialize the PostgreSQL data directory:
`sudo su - postgres -c /usr/lib/postgresql/15/bin/initdb -D /var/lib/postgresql/15/main`
Replace 15 with our version
4. **Start PostgreSQL Server:** Use this command to start the PostgreSQL server:
`sudo systemctl start postgresql`

macOS:

1. **Download PostgreSQL Installer:** Download the latest PostgreSQL installer from the official website: <https://www.postgresql.org/download/macosx/>

2. **Run Installer:** Double-click the downloaded installer file and follow the on-screen instructions. During installation, you can choose to install pgvector as an additional component.
3. **Initialize PostgreSQL:** After installation, run the following command in our terminal to initialize the PostgreSQL data directory:
`/Library/PostgreSQL/15/bin/initdb -D /Library/PostgreSQL/15/data`
Replace 15 with our version
4. **Start PostgreSQL Server:** Use this command to start the PostgreSQL server:
`launchctl load /Library/LaunchDaemons/com.postgresql.postgresql.plist`

WINDOWS:

1. **Download PostgreSQL Installer:** Download the latest PostgreSQL installer for Windows from the official website: <https://www.postgresql.org/download/windows/>
2. **Run Installer:** Double-click the downloaded installer file and follow the on-screen instructions. During installation, you can choose to install pgvector as an additional component.
3. **Initialize PostgreSQL:** After installation, navigate to the PostgreSQL pgAdmin stack builder tool and follow the prompts to initialize the PostgreSQL data directory.
4. **Start PostgreSQL Server:** The installer should automatically configure the PostgreSQL server to start as a Windows service. You can verify and manage the service from the Windows Services console.

ENABLING PGVECTOR EXTENSION:

Once PostgreSQL is installed, you can enable the pgvector extension in our database using the following command within psql (PostgreSQL command line interface):

```
psql -U username -d database_name -c "CREATE EXTENSION vector;"
```

- Replace username with our PostgreSQL username.
- Replace database_name with the name of the database where you want to enable pgvector.

VERIFYING INSTALLATION:

After enabling the extension, you can verify its installation by running the following command in psql:

```
psql -U username -d database_name -c "\dx;"
```

This should list the installed extensions, including pgvector if the installation was successful.

PYTHON & PACKAGE INSTALLATION

PYTHON

Here are the installation steps for Python on different operating systems:

Windows:

1. **Download the installer:** Head over to the official Python downloads page: <https://www.python.org/downloads/windows/>
2. **Choose the appropriate installer:** Select the latest version under "Windows x86-64 executable installer" (assuming you have a 64-bit system). There's also a 32-bit version if needed.
3. **Run the installer:** Double-click the downloaded installer file and follow the on-screen instructions. **Crucially, check the box for "Add Python 3.x to PATH" during installation.** This ensures you can access Python from our command prompt or terminal by simply typing python or python3.
4. **Verification:** Open a command prompt (search for "cmd" in the Start menu) and type python --version or python3 --version. You should see the installed Python version displayed.

macOS:

1. **Using Homebrew (recommended):** Install Homebrew, a popular package manager for macOS, by following the instructions here: <https://docs.brew.sh/Installation>
Once Homebrew is installed, open a terminal and run the following command:
brew install python
2. **Using the official installer (alternative):** Download the macOS installer from the official Python downloads page (same as Windows). Double-click the downloaded installer file and follow the on-screen instructions.

LINUX:

The installation process varies depending on our Linux distribution. Here are some general guidelines:

1. **Update package lists:** Use the appropriate command for our distribution to update package information (e.g., sudo apt update for Ubuntu/Debian, sudo yum update for RedHat/CentOS).
2. **Install Python:** Use the package manager command specific to our distribution:
 - Ubuntu/Debian: sudo apt install python3
 - RedHat/CentOS: sudo yum install python3
 - Fedora: sudo dnf install python3
 - Other distributions: Consult our distribution's documentation for the appropriate command.
3. **Verification:** Open a terminal and type python3 --version. You should see the installed Python version displayed.

PYTHON PACKAGE INSTALLATION

Installation Method:

We will use pip, the recommended package installer for Python. You can usually run pip commands directly in our terminal.

Following is the list of packages that need to be installed as a part of data loading and RAG implementation process:

- **PyMuPDFLoader:** Loads and extracts text from PDF documents.
Installation: pip install PyMuPDFLoader
- **langchain_community:** Collection of tools for natural language processing tasks.
Installation: pip install langchain_community
- **bs4:** BeautifulSoup - parses HTML and XML documents.
Installation: pip install bs4
- **psycopg2-binary:** Enables interaction with PostgreSQL databases (includes pre-compiled binaries).
Installation: pip install psycopg2-binary
- **pgvector:** Provides vector search capabilities for PostgreSQL databases.
(assuming PostgreSQL is already installed): pip install pgvector

BACKEND DEVELOPMENT

DEVELOPMENT ENVIRONMENT

Setting up the development environment.

Prerequisites:

Before you start, ensure you have the following installed:

- Python 3.8 or higher
- pip (Python package installer)
- A code editor or IDE (e.g., VSCode, PyCharm)

Create a Virtual Environment

A virtual environment isolates our project dependencies, preventing conflicts with other projects. "python -m venv venv". Activate the virtual environment: "venv\Scripts\activate"

Install Required Python Packages:

1. Create a requirements.txt file in our project directory and add the following packages:
2. "Flask==2.2.3 psycopg2-binary==2.9.5 flask_sqlalchemy==3.0.2 torch==2.0.0"
3. Install the packages using pip : "pip install -r requirements.txt"

FRONTEND DEVELOPMENT

Below is the list of technologies used in frontend development:

- *HTML*
- *CSS*
- *Java Script*
- *PHP – A backend technology for the above front–end technologies.*

DEVELOPMENT ENVIRONMENT

INSTALLING HTML, CSS, AND JAVASCRIPT: A MISCONCEPTION

HTML, CSS, and JavaScript don't require installation like traditional software. They are text-based languages that can be written and executed using any text editor or code editor.

WHAT YOU NEED:

- **A text editor or code editor:** This is where you will write your HTML, CSS, and JavaScript code. Popular options include:
 - ❖ Visual Studio Code
 - ❖ Sublime Text
 - ❖ Atom
 - ❖ Notepad++ (for Windows)
 - ❖ TextEdit (for macOS)
- **A web browser:** To see the results of your code, you will need a web browser like Chrome, Firefox, Safari, or Edge.

HOW TO START:

1. **Create a new file:** Open your chosen text editor and create a new file with a .html extension (e.g., index.html).
2. **Write your HTML code:** Start typing HTML elements within the file.
3. **Create CSS file (optional):** Create a new file with a .css extension to style your HTML elements.
4. **Create JavaScript file (optional):** Create a new file with a .js extension to add interactivity to your webpage.
5. **Open the HTML file:** Open the index.html file in your web browser to see the results.

No installations are necessary. You simply need a text editor and a web browser to start creating web pages.

INSTALLING XAMPP ON DIFFERENT OPERATING SYSTEMS

WHAT IS XAMPP?

XAMPP is a free and open-source software suite that consists of the Apache web server, MySQL (or MariaDB) database, PHP, and Perl. It's a popular choice for local web development environments.

INSTALLATION STEPS

WINDOWS

- **Download:** Visit the official XAMPP website (<https://www.apachefriends.org/index.html>) and download the appropriate installer for your Windows system (32-bit or 64-bit).
- **Run the installer:** Double-click the downloaded file to start the installation process.
- **Choose components:** Select the components you want to install (Apache, MySQL, PHP, etc.). It is usually recommended to install all components for a complete development environment.
- **Installation directory:** Choose the installation directory for XAMPP.
- **Complete installation:** Follow the on-screen instructions to finish the installation.
- **Start XAMPP:** After installation, start the Apache and MySQL services from the XAMPP control panel.

MACOS

- **Download:** Visit the official XAMPP website and download the macOS version of XAMPP.
- **Open the DMG file:** Double-click the downloaded DMG file to open it.
- **Drag and drop:** Drag the XAMPP icon to the Applications folder.
- **Start XAMPP:** Open the XAMPP application and start the Apache and MySQL services.

LINUX

Note: The installation process varies slightly depending on your Linux distribution. Here's a general guide:

- **Download:** Download the appropriate XAMPP package for your Linux distribution from the official website.
- **Extract:** Extract the downloaded archive to your desired location.
- **Start XAMPP:** Navigate to the extracted directory and start the Apache and MySQL services. The exact commands will depend on your Linux distribution. For example, on Ubuntu/Debian, you might use:
`sudo ./xampp start`

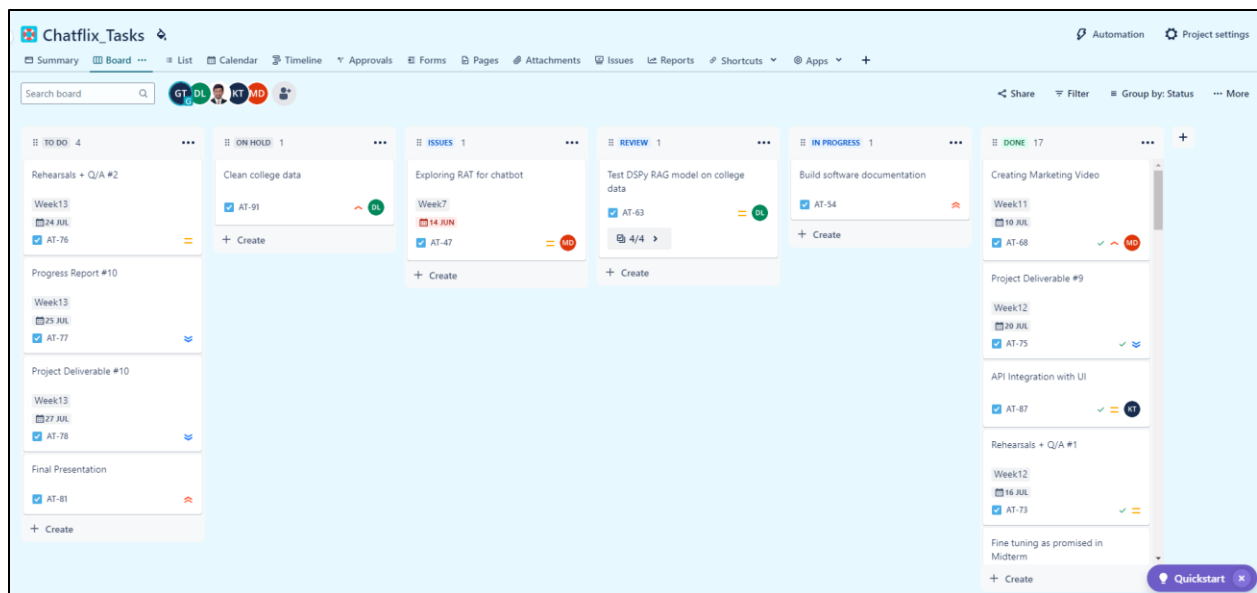
ADDITIONAL CONSIDERATIONS

- **Firewall:** Ensure that your security system allows access to ports 80 (HTTP) and 443 (HTTPS) for Apache, and 3306 for MySQL.
- **Configuration:** You might need to configure XAMPP settings for specific needs, such as PHP settings or MySQL databases.
- **Security:** It is crucial to secure your XAMPP installation, especially if you are exposing it to the internet. Change default passwords, keep software updated, and follow security best practices.

By following these steps, you should have XAMPP installed and ready to use on your operating system.

PROJECT MANAGEMENT

There are various project management tools in the market, but there is one tool which is an industry standard and stands out more, it's JIRA. We have used JIRA for our project management



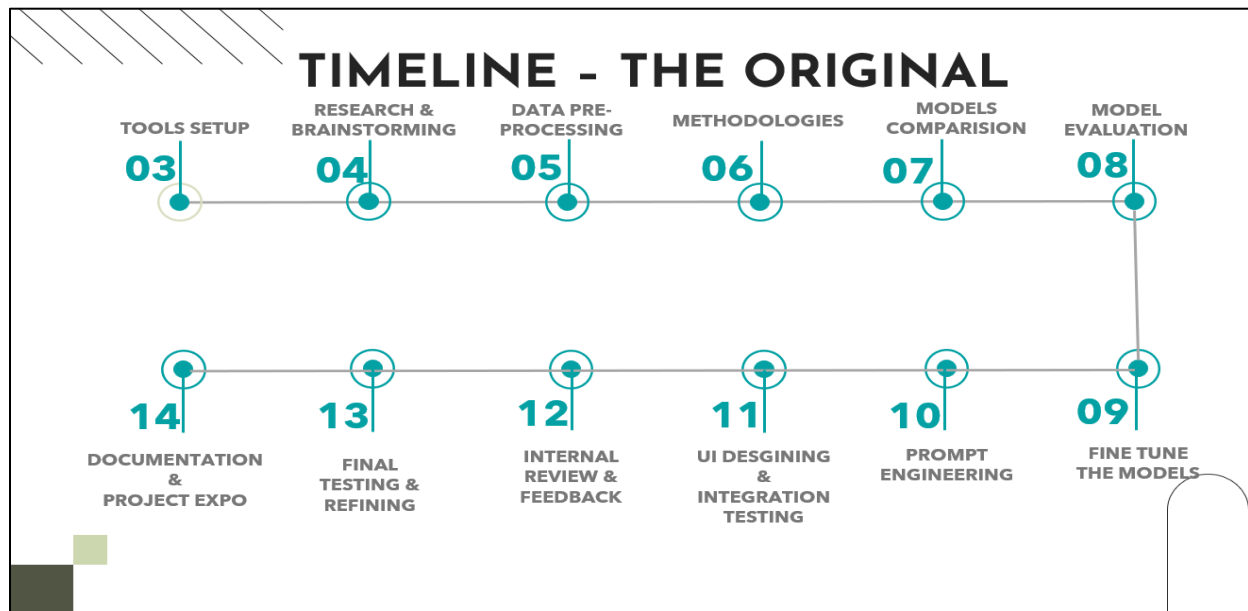
You can find the project management details from the below URL for our reference:

<https://ace-9.atlassian.net/>

PROJECT TIMELINE - THE ORIGINAL

Below is the original timeline that we have planned as a part of the AIP program:

Project Timeline – Original



The timeline is divided into two phases, representing two iterations or stages of the project. Each phase consists of several key activities, with each activity assigned a specific week for completion.

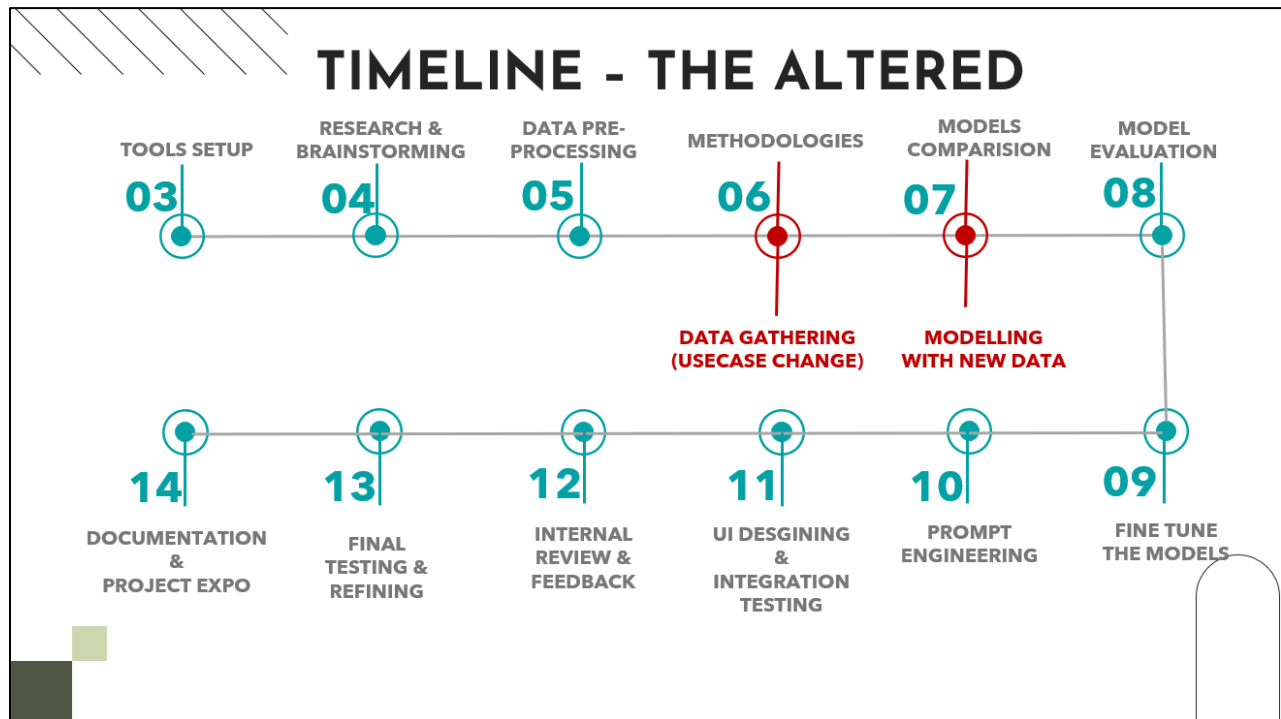
Phase 1 (Weeks 03-08)

- **Week 03:** Tools Setup
- **Week 04:** Research & Brainstorming
- **Week 05:** Data Pre-Processing
- **Week 06:** Methodologies
- **Week 07:** Models Comparison
- **Week 08:** Model Evaluation

Phase 2 (Weeks 09-14)

- **Week 09:** Fine Tune the Models
- **Week 10:** Prompt Engineering
- **Week 11:** UI Designing & Integration Testing
- **Week 12:** Internal Review & Feedback
- **Week 13:** Final Testing & Refining
- **Week 14:** Documentation & Project Expo

PROJECT TIMELINE - ALTERED TIMELINE



The provided image outlines a revised project timeline, necessitated by a shift in the project's use case.

The timeline is divided into two phases, each represented by a horizontal line with numbered markers indicating specific weeks.

Phase 1 (Weeks 03-08):

- **Foundation:** Begins with tool setup, research, and data preprocessing.
- **Methodology and Model Development:** Includes establishing methodologies, comparing models, and evaluating their performance.

Phase 2 (Weeks 09-14):

- **Adaptation:** Introduces a new phase due to a use case change, focusing on data gathering and modeling with the new data.
- **Refinement and Finalization:** Encompasses fine-tuning models, prompt engineering, UI design, internal review, testing, and documentation.

KEY CHANGES:

The most significant alteration is the insertion of a new phase between the initial model evaluation and the subsequent refinement stages. This suggests a mid-project adjustment to accommodate

a revised project goal or approach. This change impacts on the overall timeline, pushing back the final project deliverables.

The timeline emphasizes a structured approach to project execution, incorporating flexibility to adapt to changes in project requirements.

TEAM STRUCTURE FOR CHATBOT DEVELOPMENT

Our team structure is well-suited for a chatbot development project. Let's break down the roles and their typical responsibilities:

ROLE BREAKDOWN

- **Frontend Developer: Mayur Donwalkar**
 - ❖ Designs and develops the user interface for the chatbot.
 - ❖ Handles the integration of the chatbot with various platforms (web, mobile, messaging apps).
 - ❖ Ensures a seamless user experience.
- **AI Engineer: Deep Lad**
 - ❖ Convert documents and knowledge base content into high-dimensional vectors using embedding models and store them in vector database.
 - ❖ Choose appropriate pre-trained language models to build RAG with CoT and fine-tune these models based on the specific needs of the chatbot.
 - ❖ Integrate the model with the vector database and evaluate model's performance using different metrics and adjust parameters as needed.
 - ❖ Assess the performance of the chatbot in terms of response time and accuracy.
- **Data Scientist: Keyur Thakkar**
 - ❖ Develops and trains machine learning models for natural language processing (NLP) tasks.
 - ❖ Prepares and processes data for model training.
 - ❖ Collaborates with the backend team to integrate models into the chatbot.
- **Data Engineer: Girish Sai Thiruvithula**
 - ❖ Builds and maintains the data infrastructure for the chatbot.
 - ❖ Collects, cleans, and transforms data for the data scientist.
 - ❖ Ensures data quality and availability.
- **Project Manager: Girish Sai Thiruvithula**
 - ❖ Oversees the entire project, sets goals, and manages timelines.
 - ❖ Coordinates team efforts and resolves conflicts.
 - ❖ Tracks progress and communicates with stakeholders.
 - ❖ Manages project budget and resources.
- **AI Engineer:**
 - ❖ Model Development: Designing, training, and fine-tuning AI models using machine learning and deep learning techniques.
 - ❖ Algorithm Implementation: Translating theoretical algorithms into efficient code.
 - ❖ Model Evaluation: Assessing model performance using appropriate metrics and techniques.

Note: While these roles provide a general framework, it's important to recognize that team members often collaborated across roles, contributing to various aspects of the project. This cross-functional approach fostered a deeper understanding of the project and led to innovative solutions.

RISK MANAGEMENT

TIMELINE ADHERENCE:

- Risk of project delays due to unforeseen challenges or resource constraints.
- Risk of scope creep impacting project timelines.
- Mitigation: Regular project reviews, agile methodologies, and contingency buffers in the timeline.

USE CASE CHANGES:

- Risk of additional project scope due to evolving requirements.
- Risk of impact on existing project timeline and budget.
- Mitigation: Flexible project planning, regular stakeholder communication, and change management processes.

During the project timelines, the risks we faced and resolved by implementing a comprehensive risk management approach, the project team can proactively address potential challenges and increase the likelihood of project success.

To know about the project management details refer the following JIRA URL:

<https://ace-9.atlassian.net/jira/core/projects/AT/board>

PORTFOLIO WEBSITE

To know more about us you can navigate to our website portfolio using the below URL:

<http://team-chatflix.wuaze.com/?i=1>

REFERENCES AND RESOURCES

Reference Links

<https://www.analyticsvidhya.com/blog/2022/02/a-simple-guide-to-rasa-3-x/>

<https://rasa.com/docs/rasa/>

<https://medium.com/@pinakdatta/building-and-deploying-a-chatbot-from-scratch-using-rasa-framework-a-complete-guide-for-beginners-4a5f87c40dda>

<https://learn.microsoft.com/en-us/azure/bot-service/bot-service-overview?view=azure-bot-service-4.0>

<https://smartbot360.com/documentation/create-flow-diagram-bot>

<https://azure.microsoft.com/en-us/pricing/details/cognitive-services/language-service/>

<https://github.com/RiyaadHossain/Digital-Cow-Hut-Backend>

<https://flask.palletsprojects.com/en/3.0.x/>

https://api.python.langchain.com/en/latest/langchain_api_reference.html

<https://github.com/pgvector/pgvector>

<https://bugbytes.io>

<https://ollama.com>

<https://github.com/ollama/ollama>

<https://python.langchain.com/v0.2/docs/integrations/lms/ollama/>

<https://www.datacamp.com/blog/dspy-introduction>

<https://qdrant.tech>

<https://python.langchain.com/v0.2/docs/tutorials/rag/>

https://python.langchain.com/v0.1/docs/use_cases/question_answering/